

基于鲁棒集合估计与滚动联合规划的无线电干扰源自主搜索与清除

摘 要

在无线电干扰源的定向测量和清除中, 为了尽快消除不良影响, 因此需要找到一种精确定位和快速清除干扰源的策略. 本文基于覆盖搜索、集合定位、滚动规划、保证清除构建适当的数学模型, 实现了对干扰源的自动搜索、定位和清除. 并且赋予了我们模型不受干扰源区域尺度的能力, 展示了模型一定程度上的泛化性.

针对问题一: 我们先通过不同监测点有界方位误差的楔形区域的交集得到定位区域, 进而用**半平面交法**得到多边形的顶点集合, 再通过**旋转卡壳法**获得凸多边形直径. 结合 Jung 定理与实际参数反例构造从理论和实际两方面说明以定位区域直径为直径的圆不一定能覆盖此定位区域的结论.

针对问题二: 为确定第二检测位置, 本文将第一次测量得到的区域作为先验集合, 以第二次检测后的交会区域直径衡量定位质量. 同时利用有效接收半径这一参数约束, 得到一块由两个圆盘交集构成的候选区, 并用**确定性全局优化中的几何分支定界思想**^[1] 在该区域内最小化最坏后验定位区域直径. 但由于题目约束较少情况较多, 我们得到基准情形最优检测点 (843.365, ± 545.021), 与四种特殊边界情况 (见正文), 以及不考虑第二次必须接收信号条件下的一个全局最优点 (970.926, ± 494.980).

针对问题三: 由于加入了路径规划与移动成本为完成全向干扰的自动搜索、定位与清除, 本文构造了由原点和六个外围站组成的**七站覆盖结构**, 保证在最坏距离 993.814m 的情况下完整覆盖任务区域. 结合**有界误差集合定位**与保证清除判据, 将扫描、检测清除任务统一纳入滚动规划, 对于小规模任务采取**子集动态规划**, 大规模任务采用**束搜索结合剩余资源监督**, 并且采用剩余覆盖扫描和蛇形遍历清除保证完成. 测试了 140 场分层实验全部清除完毕, 平均清除时间 235.8s, 并且随着干扰源增加有下降趋势.

针对问题四: 考虑定向干扰源存在发射方向限制, 本文在问题三基础上建立**位置-方向联合覆盖模型**, 并利用凸包条件构造**二十一点扫描布局**. 随后将剩余扫描与目标定位、清除任务统一纳入**混合整数线性规划**, 并根据实时反馈**滚动更新路径**; 对难以直接清除的目标采用 25 m 格点遍历. 100 组随机场景均完成全部干扰源清除, 平均定位清除时间为 449.58 s/源, 验证了模型的有效性与稳定性.

关键词: 混合整数线性规划; 在线滚动联合决策; 集合定位; 路径规划; 位置-方向联合覆盖模型

一、问题重述

1.1 问题背景

无线电干扰源的快速定位与清除是无线电频谱管理中的重要问题。在复杂电磁环境下存在一定安全风险。为提高干扰源搜索与处置效率，题目提出利用搭载测向机、光学探测仪和激光清除装置的机器狗，在圆形目标区域内完成干扰源的自动检测、定位与清除。机器狗可在不同位置对干扰源进行测向，但是带有一定误差的示向度，因此需要通过多次检测形成交会定位区域；当机器狗足够接近干扰源时，可进一步利用光学设备完成精确定位与清除。我们需要围绕定位区域计算、检测点选择以及搜索清除策略建立相应的数学模型和算法。

1.2 问题提出

基于上述背景，要求建立数学模型解决以下问题。

问题 1：已知若干检测点的位置及同一干扰源在各检测点处的示向度，建立交会定位区域的计算方法，求取该区域的直径，并判断以该直径为直径的圆能否完全覆盖定位区域。

问题 2：已知某全向干扰源在第一次检测点处的示向度，在考虑测向误差及有效接收范围的条件下，确定第二检测点的候选区域，并设计第二检测点的选择策略，以尽可能改善二次检测后的定位效果。

问题 3：当目标区域内仅存在全向干扰源，且干扰源总数未知时，设计机器狗从原点出发的自动搜索、定位与清除策略及相应算法，并通过模拟器测试，以被清除干扰源比例和平均定位清除时间评价策略效果。

问题 4：在目标区域同时存在全向干扰源和定向干扰源的情况下，进一步考虑定向辐射带来的检测限制，改进机器狗的搜索、定位与清除策略，并通过模拟器验证算法的有效性。

二、问题分析

2.1 问题一的分析

对于问题一，由于测向结果存在 $\pm 1^\circ$ 的误差，单次示向只能确定一个扇形可行区域，因此需要综合多个检测点的测向信息，通过求取各可行域的公共交集确定干扰源的定位区域。为适应不同数量的检测点，我们将各测向约束转化为半平面约束，并利用半平面交算法获得凸多边形顶点集合。在得到定位区域后，用旋转卡壳算法计算凸多边形直径。最后思考最小包围圆与区域直径之间的关系来分析以该直径为直径的圆能否覆盖整个定位区域，并通过具体算例验证结论。

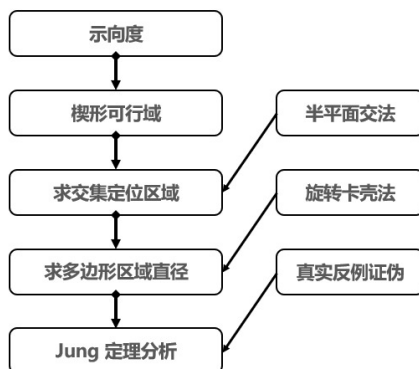


图 1 问题一交会定位、区域直径计算及覆盖性分析流程

2.2 问题二的分析

由于一次测向只能将干扰源限制在带有测向误差的可行区域内，因此需要合理选择第二检测点进一步缩小定位范围。考虑到真实目标位置和有效接收半径均未知，本文首先确定能够保证第二次获得有效反馈的候选区域，再以第二次观测后的最坏定位区域直径作为评价指标，在候选区域内优化第二检测点位置。考虑到初始检测点是随机的，我们根据第一次可行域是否受到目标圆边界截断分别处理：未截断情形可利用平移、旋转不变性转化为标准模型；截断情形则采用连续二维搜索与分支定界方法求解，并通过上下界差距来评价推荐点的最优性间隙。

2.3 问题三的分析

问题三需要在干扰源数量、位置与有效接收半径均未知的条件下，联合完成频道搜索、目标定位与路径规划。本文先利用原点和六个外围扫描站保证目标圆域的覆盖搜索，再用带有界误差的示向约束持续更新各目标的位置集合；随后把剩余扫描、补充检测和保证清除统一为待访问任务，通过滚动规划减少机器狗的往返距离，并设置蛇形遍历清除与剩余资源可行性检查保证任务能够完成。

2.4 问题四的分析

本问沿用问题三的虚拟时间代价和平均定位清除时间指标，不再重复列式。算法仍采用“覆盖搜索-集合定位-滚动规划-保证清除”的主线，但搜索层改为 21 点位置-方向联合覆盖，规划层改为“剩余覆盖路径 MILP + 任务执行方案-访问顺序联合 MILP”。扫描过程中一旦发现目标，即将其定位与清除任务加入当前路线；局部定位达到保证清除条件时直接清除，达到 8 次测量或 4 次失败清除时转入 25 米格点清除。

三、模型假设

- 1、假设干扰源状态在单次任务过程中保持不变，即干扰源的位置、工作频道、有效接收半径以及定向干扰源的发射方向在一次测试过程中不发生变化。
- 2、假设机器狗的定位与运动控制准确，忽略定位误差、姿态变化及加减速过程，将机器狗在目标区域内的移动视为平面上的直线运动，并按照题目给定速度计算移动时间。
- 3、假设测向误差满足题目给出的确定性误差界，即正常示向结果的真实方向位于测得方向的 $\pm 1^\circ$ 范围内，除该有界误差外不再考虑额外随机测量误差。
- 4、假设各类检测与清除反馈均准确可靠，即 `direction`、`near`、`no_signal` 以及清除成功或失败等反馈均严格符合题目规定的距离和方向条件。
- 5、假设无线电传播条件在任务过程中保持稳定。全向干扰源的有效接收范围仅由干扰源与机器狗之间的距离决定；除题目规定的定向辐射限制外，忽略遮挡、多径传播及环境衰减等因素的影响。
- 6、假设不同干扰源之间相互独立，对某一频道进行检测、定位或清除时，其他干扰源不会改变该目标对应的测向结果和有效接收条件。

四、符号说明

表 1 主要符号说明

符号	说明	单位
$S_i = (x_i, y_i)$	第 i 个检测点的位置	m
θ_i	第 i 个检测点测得的干扰源示向角	°
δ	测向误差半宽, 本文取 $\delta = 1^\circ$	°
$W(S, \theta, \delta)$	检测点 S 在示向角 θ 及误差界 δ 下形成的楔形可行域	—
$\mathcal{A}$	干扰源可能存在的目标区域	—
P	综合多次测向信息后得到的干扰源位置可行区域	—
V_i	定位区域凸多边形的第 i 个顶点	m
$D(P)$	可行区域 P 的直径, 即区域内任意两点之间的最大距离	m
Ω_1	第一次检测后得到的干扰源位置可行集合	—
C_{sig}	能够保证第二次检测获得有效信号的候选区域	—
$U(s)$	选择位置 s 作为第二检测点时的最坏后验定位区域直径	m
V^*	第二检测点优化问题得到的最优最坏定位直径	m
N	场景中实际干扰源数量	个
L	机器狗执行任务过程中的总移动距离	m
T	一次任务所消耗的总虚拟时间	s
J	平均定位清除时间, 即 $J = T/N$	s/目标
C	根据当前目标可行区域构造的保证清除区域	—
R_s	保证清除判定采用的安全半径	m

五、问题一：旋转卡壳法求解定位区域直径及其覆盖性分析

5.1 模型建立与相关算法概念解析

5.1.1 有界方位误差与交会定位

查阅相关文献发现, Tokekar 和 Isler 在研究有界方位误差传感器定位问题时, 同样根据测量误差范围, 将目标可能位置表示为以检测点为顶点的楔形区域, 并通过多个楔形区域的交集描述最终的位置不确定区域^[2]. 这一处理方式与本题给出的测向误差特征一致.

具体而言, 本题示向度误差满足 $\pm 1^\circ$, 因此一次测向结果对应于中心示向度两侧各偏转 1° 所形成的楔形可行域. 该楔形区域可以表示为两个线性半平面的交集, 因此本身为凸集. 不同检测点产生的多个楔形区域继续求交后仍保持凸性, 若交集非空且有界, 则最终形成一个凸多边形定位区域.

5.1.2 半平面交算法

由前述有界方位误差模型可知, 每次测向对应的楔形可行域可以表示为两个半平面的交, 因此多个检测点的交会定位问题本质上是多个半平面的公共交问题.

当检测点数量较少时, 可以直接枚举各边界直线的两两交点, 再筛选满足全部约束的可行点. 该方法实现简单、在小规模数据下可快速得到结果. 但当检测点增多时, 简单枚举会产生大量无效候选交点, 因此计算量会明显增加. 为提高模型在不同检测规模下的适用性和可扩展性, 本文采用更成熟的半平面交 (Half-Plane Intersection, HPI) 算法. 具体操作详见附录 0.1

5.1.3 旋转卡壳法

对于已经得到的凸多边形, 需要进一步求区域内任意两点之间的最大距离, 即凸多边形直径. 当顶点数量较少时, 可以直接枚举全部顶点对进行计算, 该方法实现简单, 但时间复杂度为 $O(m^2)$. 但考虑到随着定位区域边界复杂度增加, 顶点数量也可能增多, 因此, 本文采用具有更好可扩展性的旋转卡壳法 (Rotating Calipers).

Shamos 提出了利用凸多边形对踵点求解直径的方法, Toussaint 随后将这一思想推广为旋转卡壳法^[3]. 其核心思想是利用两条平行支撑线沿凸多边形边界同步旋转, 仅考察可能构成最远点对的对踵点, 从而避免遍历全部顶点组合.

由于旋转过程中两个接触点均只沿凸多边形边界单向移动, 因此全部对踵点可在线性时间内完成扫描, 算法复杂度为

$$O(m).$$

相比直接枚举顶点对的 $O(m^2)$ 方法, 旋转卡壳法在顶点数量增加时具有更好的计算效率和可扩展性

5.1.4 Jung 定理

区域直径反映区域内任意两点之间的最大距离, 但并不等同于覆盖整个区域所需圆的直径. 设平面有界集合的直径为 D , 最小包围圆半径为 R_* . Jung 定理给出^[4]

$$R_* \leq \frac{D}{\sqrt{3}}.$$

另一方面, 由于集合中存在两个距离为 D 的点, 显然有 $R_* \geq \frac{D}{2}$. 因此

$$\frac{D}{2} \leq R_* \leq \frac{D}{\sqrt{3}}. \quad (1)$$

这说明以区域直径 D 为直径的圆, 其半径只有 $D/2$, 一般不能保证覆盖整个定位区域. 后文将结合实际测向数据构造凸四边形反例, 对该结论进行验证.

5.2 模型求解过程

5.2.1 利用半平面交算法对交会区域模型的建立

由题意得, 设第 i 个检测点的位置为 $S_i = (x_i, y_i)$, 测得干扰源示向度为 θ_i . 根据题意, 测向误差满足

$$\theta_i^{\text{true}} \in [\theta_i - \varepsilon, \theta_i + \varepsilon], \quad \varepsilon = 1^\circ. \quad (2)$$

定义示向度误差区间两侧的单位方向向量

$$\mathbf{u}_i^- = \begin{pmatrix} \cos(\theta_i - \varepsilon) \\ \sin(\theta_i - \varepsilon) \end{pmatrix}, \quad \mathbf{u}_i^+ = \begin{pmatrix} \cos(\theta_i + \varepsilon) \\ \sin(\theta_i + \varepsilon) \end{pmatrix}. \quad (3)$$

根据有界方位误差模型^[2], 第 i 次测向对应的楔形可行域可以表示为

$$W_i = \{X : \mathbf{u}_i^- \times (X - S_i) \geq 0, \quad \mathbf{u}_i^+ \times (X - S_i) \leq 0\}. \quad (4)$$

因此, n 个检测点共同确定的交会定位区域为

$$P = \bigcap_{i=1}^n W_i. \quad (5)$$

每个楔形区域由两个半平面构成, 因此式 (5) 本质上是 $2n$ 个半平面的公共交问题. 本文采用半平面交算法求解该可行区域^[5,6]. 处理全部半平面后, 若交集非空且有界, 则队列中保留下来的边界即为定位区域的有效边界. 设其依次为 $L_0, L_1, \dots, L_{m-1}$, 则相邻有效边界的交点

$$V_i = L_i \cap L_{i+1}, \quad i = 0, 1, \dots, m-1 \quad (6)$$

依次构成凸多边形定位区域, 其中下标按模 m 计算. 故最终有

$$P = \text{conv}\{V_0, V_1, \dots, V_{m-1}\}. \quad (7)$$

5.2.2 基于旋转卡壳法的区域直径求解

在 (7) 式中我们可以得到构成多边形的点的集合，定义其直径为区域内任意两点之间的最大欧氏距离：

$$D(P) = \max_{X, Y \in P} \|X - Y\|_2. \quad (8)$$

根据凸多边形最远点必属于对踵点对的性质，本文采用旋转卡壳法搜索定位区域的最远点对^[3]。

对于当前边 $V_i V_{i+1}$ ，定义

$$\mathcal{A}(i, j) = |(V_{i+1} - V_i) \times (V_j - V_i)|, \quad (9)$$

其中 $\mathcal{A}(i, j)/2$ 为三角形 $\triangle V_i V_{i+1} V_j$ 的面积。在底边固定时，该面积与 V_j 到当前边所在直线的垂直距离成正比，因此可以利用面积变化确定对侧支撑顶点。

当 $\mathcal{A}(i, j+1) > \mathcal{A}(i, j)$ 时，令

$$j \leftarrow j + 1, \quad (10)$$

直至面积不再增加，此时 V_j 即为当前边对应的对踵顶点。随后依次旋转支撑边，并继续搜索下一组对踵点。

对全部对踵点对计算欧氏距离并取最大值，得到定位区域直径

$$D(P) = \max_{\text{对踵点}(V_i, V_j)} \|V_i - V_j\|_2. \quad (11)$$

记取得最大距离的一对顶点为 A 、 B ，则

$$\|A - B\|_2 = D(P). \quad (12)$$

当出现 $\mathcal{A}(i, j+1) = \mathcal{A}(i, j)$ 时，应同时保留 V_j 与 V_{j+1} 所形成的候选点对，以避免平行边情形下遗漏真正的最远点对。

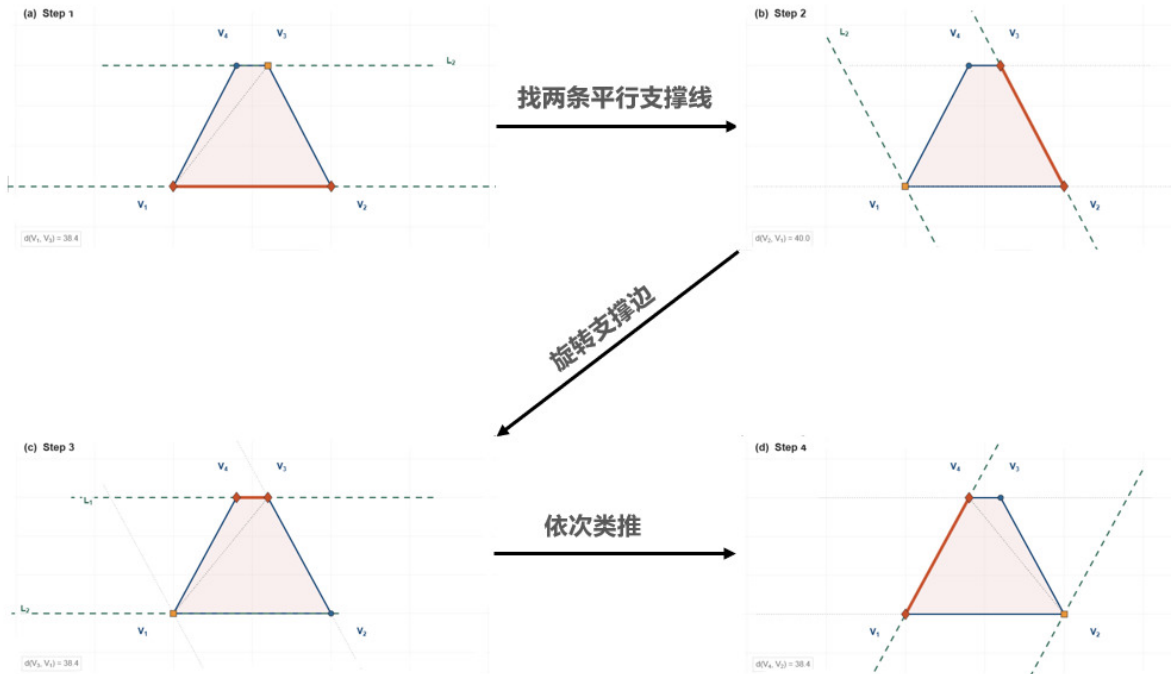


图 2 旋转卡壳法求解直径具体流程

5.2.3 基于 Jung 定理的圆覆盖性分析与凸四边形反例

由前文 Jung 定理可知, 对于直径为 D 的平面有界区域, 其最小包围圆半径 R_* 满足

$$\frac{D}{2} \leq R_* \leq \frac{D}{\sqrt{3}}.$$

即

以定位区域直径为直径的圆不一定能够覆盖整个定位区域. (13)

为进一步直观验证这一结论, 本文构造一组满足 $\pm 1^\circ$ 示向度误差约束的测向实例. 四个检测点产生的楔形可行域经半平面求交后, 得到凸四边形定位区域

$$P = \text{conv}\{V_1, V_2, V_3, V_4\},$$

其中四个顶点分别为

$$\begin{aligned} V_1 &= (-20, 0), & V_2 &= (20, 0), \\ V_3 &= (4, 30), & V_4 &= (-4, 30). \end{aligned} \quad (14)$$

利用旋转卡壳法计算该凸四边形的最远点对, 得到区域直径为 $D = \|V_1 - V_2\|_2 = 40$ m. 因此可取直径端点

$$A = V_1 = (-20, 0), \quad B = V_2 = (20, 0).$$

若存在以 D 为直径且能够覆盖整个定位区域的圆, 由于该圆必须同时包含距离恰为 D 的端点 A 、 B , 其圆心只能为二者中点

$$M = \frac{A + B}{2} = (0, 0), \quad (15)$$

相应半径为

$$R = \frac{D}{2} = 20 \text{ m}. \quad (16)$$

计算上方两个顶点到圆心 M 的距离, 有

$$\begin{aligned} \|V_3 - M\|_2 &= \sqrt{4^2 + 30^2} = \sqrt{916} \approx 30.27 \text{ m}, \\ \|V_4 - M\|_2 &= \sqrt{(-4)^2 + 30^2} = \sqrt{916} \approx 30.27 \text{ m}. \end{aligned} \quad (17)$$

由于

$$30.27 \text{ m} > \frac{D}{2} = 20 \text{ m}, \quad (18)$$

故顶点 V_3 、 V_4 均位于该直径圆之外. 因此, 以定位区域直径 D 为直径的圆无法覆盖整个凸四边形定位区域, 从而给出了式 (17) 的一个明确反例.

该构造过程如图 3 所示. 图中首先由多个带有 $\pm 1^\circ$ 测向误差的楔形区域进行交会定位, 再提取所得凸四边形定位区域, 最后绘制以区域直径为直径的圆, 可以直观观察到定位区域的上部顶点位于圆外.

5.3 问题一结论与数学模型

对于第一问, 首先依据有界方位误差模型, 将每次示向度观测转化为楔形可行域, 并通过多个可行域求交得到凸多边形定位区域^[2]; 随后利用旋转卡壳法搜索凸多边形的对踵点, 得到定位区域直径^[3]; 最后结合 Jung 定理及实际凸四边形反例, 说明以定位区域直径为直径的圆并不能保证覆盖整个定位区域^[4]. 数学模型如下:

$$\left\{ \begin{array}{ll} \text{交会定位区域:} & P = \bigcap_{i=1}^n W_i, \\ \text{定位区域直径:} & D(P) = \max_{\text{对踵点}(V_i, V_j)} \|V_i - V_j\|_2, \\ \text{圆覆盖关系:} & \frac{D(P)}{2} \leq R_* \leq \frac{D(P)}{\sqrt{3}}. \end{array} \right.$$

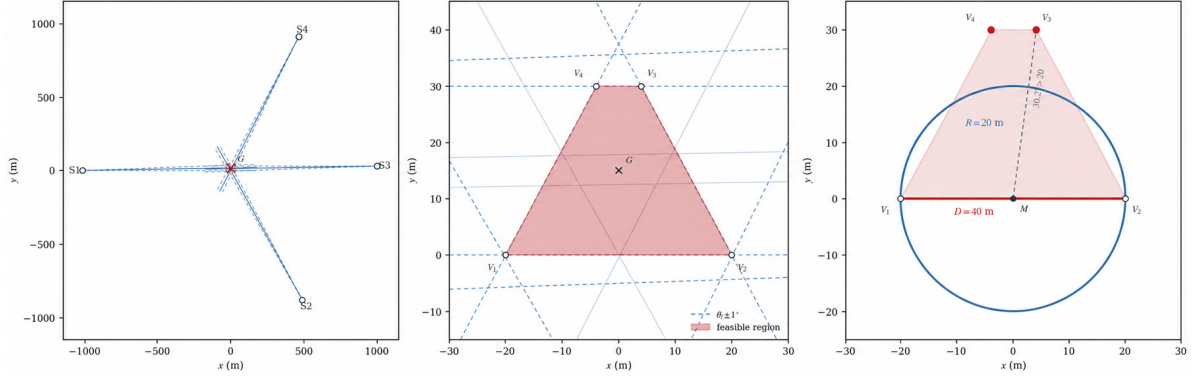


图 3 凸四边形定位区域的构造及直径圆覆盖性反例：(a) 有界误差下的测向楔形；(b) 半平面交得到的定位区域；(c) 以区域直径为直径的圆无法完全覆盖定位区域。

六、问题二：基于分支定界和最坏定位直径的优化策略

6.1 模型建立与概念定义

6.1.1 由理想交会几何到鲁棒性能指标

问题二要求我们已知第一次检测位置及其示向度, 需要进一步选择第二检测点. 已有的 Doğançay 和 Hmam 基于 Fisher 信息矩阵对该问题的研究表明, 在双传感器与目标距离给定的条件下, 正交的观测方向对应最优的观测效果^[7].

但该结论建立在目标距离和观测几何已知的理想条件下, 而本题中真实目标距离、有效接收半径及第二检测点与目标的距离均未知, 因此不能直接据此确定第二检测点.

本题的测向误差明确满足连续的 $\pm 1^\circ$ 确定性误差界, 参考有界误差下的集合成员估计思想 (set-membership estimation)^[8], 与其围绕某个名义目标位置优化交会角, 更自然的做法是保留所有与观测及误差界相容的目标位置, 并在该集合上考虑最不利情形去进行优化. 结合问题一对“定位区域直径”的定义, 本文采用**最坏后验定位区域直径**作为第二检测点的性能指标, 使问题二的评价标准与问题一保持一致.

考虑到有效接收半径仅知位于 $[1000, 1500]$ m 内, 本文进一步要求第二检测点对所有与首次观测相容的目标均能够获得有效反馈, 并在该候选区域内最小化最坏后验定位直径, 从而形成有界不确定集合上的鲁棒极小极大优化^[9]. 取消该约束的情形将在第 6.3 节讨论.

记目标区域为 $\mathcal{A}$. 沿用问题一的有界方位误差模型, 以 $W(S, \theta, \delta)$ 表示检测点 S 在示向度 θ 、误差界 $\delta = 1^\circ$ 下对应的楔形可行域. 由于第一次已经正常获得示向度, 真实干扰源 g 还应满足 $5 < \|g - S_1\|_2 \leq 1500$. 因此第一次观测后的完整目标可行域为

$$\Omega_1 = \mathcal{A} \cap W(S_1, \theta_1, \delta) \cap \{g : 5 < \|g - S_1\|_2 \leq 1500\}. \quad (1)$$

由于第一次检测已经成功接收到该源, 对于任意 $g \in \Omega_1$, 其真实接收半径还必须满足 $R \geq \|g - S_1\|_2$. 结合题设 $R \geq 1000$ m, 可得与位置 g 相容的最小有效接收半径. 那么在目标位置为 g 的条件下, 与当前信息相容的最小有效接收半径为

$$L(g) = \max \{1000, \|g - S_1\|_2\}. \quad (2)$$

由此定义保证第二次有效接收的候选区域

$$C_{\text{sig}} = \bigcap_{g \in \Omega_1} \bar{B}(g, L(g)) = \{s : \|s - g\|_2 \leq L(g), \forall g \in \Omega_1\}. \quad (3)$$

其中 $\bar{B}(c, r)$ 表示以 c 为圆心、 r 为半径的闭圆盘. 式 ((3) 式) 的含义是: 对于第一次观测后仍然可能存在的任一真实目标位置, 以及与该位置相容的任一真实有效接收半径, 第二检测点都能够再次获得有效反馈. 因此, 本文将 C_{sig} 作为题目所要求的**第二检测点候选区域**; 在此基础上, 再比较候选区域内不同位置的定位性能, 以确定具体的推荐检测点.

定位区域直径 $D(\cdot)$ 沿用问题一的定义.

$$\Omega_2(s, o) = \begin{cases} \Omega_1 \cap \{g : \|g - s\|_2 \leq 5\}, & o = \text{near}, \\ \Omega_1 \cap W(s, z, \delta) \cap \{g : 5 < \|g - s\|_2 \leq 1500\}, & o = z \text{ (direction)}. \end{cases} \quad (4)$$

并对第二检测点处所有可能发生的有效反馈取最坏情况, 定义

$$U(s) = \sup_o D(\Omega_2(s, o)). \quad (5)$$

其中 o 遍历位置 s 处所有与第一次观测相容的 near 或 direction 反馈. 于是, $U(s)$ 表示机器狗选择第二检测点 s 后, 在最不利真实目标位置与最不利第二次反馈下仍可能出现的最大定位区域直径.

在确定第二检测点候选区域 C_{sig} 后, 为进一步给出候选区域内的具体选择策略, 本文求解:

$$V^* = \inf_{s \in C_{\text{sig}}, s \neq S_1} U(s). \quad (6)$$

在实际求解中, 该算法输出一个可行推荐点 s_{rec} 以及 V^* 的上下界, 并通过最优性差距评价其最优性间隙. 由此, C_{sig} 回答了“第二检测点可以选在哪里”, 即给出了题目要求的候选区域; 而 $U(s)$ 则用于比较候选区域内各位置的定位效果, 并据此确定推荐第二检测点. 下文均沿用上述 Ω_1 、 C_{sig} 、 $\Omega_2(s, o)$ 、 $U(s)$ 和 V^* 的定义

6.1.2 一般首次检测位置下的目标可行域

由于题目没有指定第一次检测位置和示向方向, 我们在此视作随机处理, 令

$$\mathbf{v}(\phi) = \begin{pmatrix} \cos \phi \\ \sin \phi \end{pmatrix}, \quad (7)$$

则第一次测向楔形中的可能目标可以参数化为

$$g = S_1 + r\mathbf{v}(\theta_1 + \varepsilon), \quad |\varepsilon| \leq \delta. \quad (8)$$

但是由于位置随机, 第一次测向楔形的区域可能会超出实际 1800m 区域, 在这之外是不会有干扰源的, 我们称该区域被截断. 对固定 ε , 由目标圆约束 $\|g\|_2 \leq 1800$ 可得关于 r 的二次不等式

$$r^2 + 2b(\varepsilon)r + \|S_1\|_2^2 - 1800^2 \leq 0, \quad (9)$$

这是一个典型二次方程, 若 $\Delta(\varepsilon) < 0$, 该方向上不存在与目标圆相容的位置; 否则可行径向区间为

$$I_\varepsilon = (5, 1500] \cap \left[-b(\varepsilon) - \sqrt{\Delta(\varepsilon)}, -b(\varepsilon) + \sqrt{\Delta(\varepsilon)} \right]. \quad (10)$$

式 ((10) 式) 同时覆盖第一次检测点位于目标圆内、圆周上甚至圆外的情况, 也能够自然描述空区间和相切退化情形.

根据目标圆是否截断完整的 5 ~ 1500 m 测向扇形, 可以将一般输入分为四类:

- 1、 $\Omega_1 = \emptyset$, 即第一次位置与示向结果不相容;
- 2、完整扇形未被目标圆边界截断;
- 3、截断且关于首次示向轴对称, 例如靠近边界时沿径向朝外或朝内;
- 4、截断且非对称, 例如边界附近的切向或斜向示向.

由式 ((10) 式) 可获得每个示向偏差 ε 对应的实际径向可行区间. 后续据此判断完整的 5 ~ 1500 m 测向扇形是否受到目标圆截断, 并分别处理未截断与截断情形.

6.2 模型求解过程与结果

6.2.1 求解过程

Step 1: 判断首次目标可行域是否被目标圆截断

由上一节得到的径向区间 I_ε ，首先判断第一次测向形成的完整 $5 \sim 1500$ m 扇形是否全部位于目标圆内。

若对所有 $|\varepsilon| \leq \delta$ 均有

$$I_\varepsilon = (5, 1500],$$

则目标圆并未对第一次测向可行域产生额外截断，称为**未截断情形**；否则根据各方向上的实际径向区间重新构造 Ω_1 ，称为**截断情形**。

这一区分十分重要：未截断情形具有平移和旋转不变性，只需求解一次标准问题；而截断情形的可行域形状随 S_1 和 θ_1 改变，需要重新进行二维优化。

Step 2: 求解未截断标准情形

对于未截断情形，以第一次检测点 S_1 为局部原点，并以第一次示向方向建立局部坐标轴。记

$$\mathbf{u} = \begin{pmatrix} \cos \theta_1 \\ \sin \theta_1 \end{pmatrix}, \quad \mathbf{n} = \begin{pmatrix} -\sin \theta_1 \\ \cos \theta_1 \end{pmatrix}.$$

经过平移和旋转后，所有未截断问题均可以转化为标准情形

$$S_1 = (0, 0), \quad \theta_1 = 0^\circ.$$

在该标准情形下，问题关于 x 轴对称，因此只需考虑第二检测点 $s = (x, y)$ 满足 $x \geq 0, y \geq 0$ 的情况。

记第一次测向楔形下边界的单位方向向量为

$$\mathbf{v}_- = \begin{pmatrix} \cos \delta \\ -\sin \delta \end{pmatrix}.$$

对于 $5 < r \leq 1000$ 的可能目标，其最小有效接收半径为 1000 m，而 $\|s - r\mathbf{v}_-\|_2^2$ 关于 r 为凸函数，因此只需检查径向区间的两个端点。

对于 $1000 \leq r \leq 1500$ ，由第一次已经成功接收可知其有效接收半径至少为 r ，相应保证接收条件为

$$\|s\|_2^2 \leq 2r s^\top \mathbf{v}_-,$$

其最严格情形仍出现在 $r = 1000$ 。

将式 ((3) 式) 的保证接收域具体化到该标准情形，上述连续目标约束可进一步解析化简为两个圆盘的交集：

$$C_{\text{sig}} \cap \{y \geq 0\} = \overline{B}(5\mathbf{v}_-, 1000) \cap \overline{B}(1000\mathbf{v}_-, 1000) \cap \{y \geq 0\}. \quad (11)$$

由此，上述两个圆盘的交集即构成标准情形上半平面的第二检测点候选区域；下半平面的候选区域可由关于首次示向轴的对称性得到。后续只需在该候选区域内比较最坏后验定位直径，并进一步确定推荐第二检测点。

对于其他未截断输入，只需将标准情形得到的局部位置 (x_0, y_0) 变换回原坐标系：

$$S_2 = S_1 + x_0 \mathbf{u} \pm y_0 \mathbf{n}. \quad (12)$$

由于平移和旋转均保持距离和角度关系不变，因此变换前后的最坏后验定位直径相同。

Step 3: 求解目标圆截断情形

当目标圆截断第一次测向扇形后， Ω_1 的形状不再与标准情形相同，此时我们直接在保证接收域 C_{sig} 上进行连续二维搜索。

首先通过数值搜索获得一个可行推荐点 $s_{\text{rec}} \in C_{\text{sig}}$ ，再依据式 ((5) 式) 对其全部可能第二次反馈进行连续覆盖验证，得到

$$U(s_{\text{rec}}) \leq UB,$$

其中 UB 为当前可行推荐点目标函数值的经连续反馈覆盖验证得到的上界。

查阅资料，在相似的问题上 Daniel Scholz 采用确定性全局优化中的几何分支定界思想 (geometric branch-and-bound)^[1] 进行解决。故我们将连续第二检测位置域递归划分为若干矩形，并针对各子区域构造性能下界与剪枝条件，对于每一个矩形：

- 1、若该矩形与 C_{sig} 无交，则直接排除；
- 2、若仍包含可行点，则在 Ω_1 中寻找一对可能目标 g, h ；
- 3、若对该矩形内任意第二检测点， g, h 都可能产生相同的第二次反馈，则这两个位置无法由该次观测区分，因此矩形内任意检测点均满足

$$U(s) \geq \|g - h\|_2.$$

由此可为每一个仍可能包含更优解的矩形建立性能下界。不断细分下界较低的区域，并排除不可能优于当前推荐点的区域，即可逐步缩小最优值所在范围。

我们借鉴区间分析中 Moore、Kearfott 等人利用保守外包覆盖连续变量取值范围的思想^[10]，将全部可能示向结果划分为连续角区间，并对每个角区整体构造后验区域外包及其直径上界。计算时将所有可能示向结果划分为连续角区间，并对每个区间整体计算后验区域的最大可能直径。

Step 4: 确定最终推荐点

设搜索得到的全局下界为 LB ，推荐检测点 s_{rec} 的最坏定位直径上界为 UB ，则有

$$LB \leq V^* \leq U(s_{\text{rec}}) \leq UB. \quad (13)$$

因此

$$0 \leq U(s_{\text{rec}}) - V^* \leq UB - LB. \quad (14)$$

因此， $UB - LB$ 给出了当前推荐点相对于连续空间全局最优值的最优性间隙。当

$$UB - LB \leq \varepsilon$$

时， s_{rec} 即为目标函数意义下的 ε -全局近优解。采用上述方法的主要优点为：

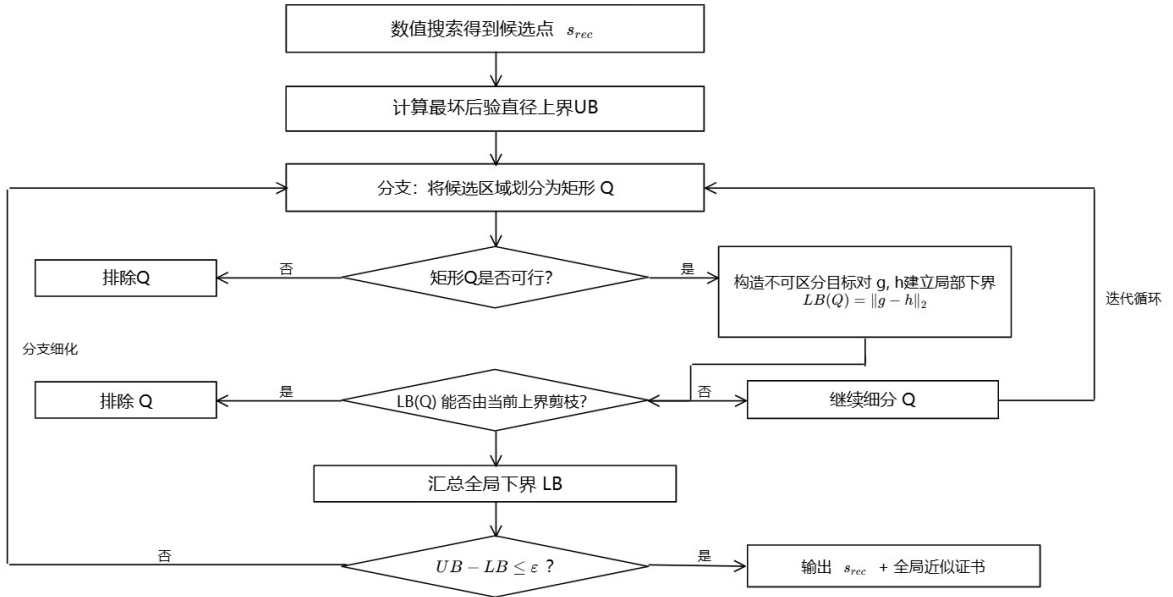


图 4 分支定界流程

- 1、未截断情形只需求解一次标准模型，其余输入可直接通过坐标变换获得结果；
- 2、对截断情形直接搜索连续二维区域，避免固定网格过粗造成的漏解；
- 3、第二次 $\pm 1^\circ$ 测向误差按连续区间处理，而不是用有限角度样本代替；
- 4、通过上下界差距可以直接评价当前推荐检测点接近全局最优的程度。

6.2.2 求解结果

(1) 基准情形与未截断情形

对于

$$S_1 = (0, 0), \quad \theta_1 = 0^\circ$$

的基准情形，计算得到推荐第二检测点

$$S_2^* \approx (843.365, \pm 545.021) \text{ m}. \quad (15)$$

其最坏后验定位区域直径满足

$$110.963 \leq V^* \leq 110.973 \text{ m}. \quad (16)$$

上下界之间仅相差 0.010 m，因此该推荐点在本文性能指标下已经十分接近连续空间中的最优结果。

这里的 0.010 m 指的是**最坏定位区域直径的性能差距**，而不是推荐点坐标与未知精确最优坐标之间的距离。

根据式 ((12) 式)，其余未截断输入均可由基准情形进行刚性转移，并继承相同的性能上下界。例如， $S_1 = (0, 100)$ 、 $\theta_1 = 90^\circ$ 与 $S_1 = (1700, 0)$ 、 $\theta_1 = 180^\circ$ 均得到与中心情形一致的最坏定位性能。

中心首测条件下第二检测点最坏后验 MEC 半径空间分布及最优位置

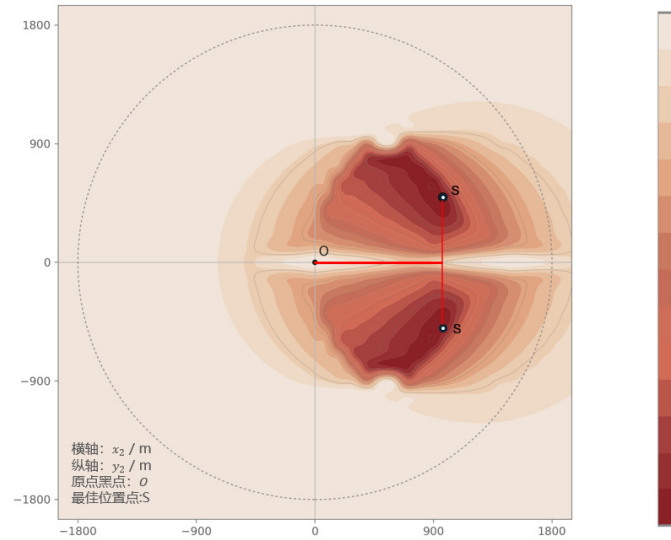


图 5 中心首测条件下保证接收域及推荐第二检测点

(2) 目标圆截断情形

为进一步考察第一次检测位置靠近目标圆边界时的情况，本文选取四类具有代表性的截断输入：边界朝外、边界切向、边界斜向以及圆外朝内。

其中，边界朝外代表较典型的轴对称截断；边界切向和边界斜向代表非对称截断；圆外朝内则说明第一次检测点即使位于目标圆之外，只要其测向扇形仍与目标区域相交，模型仍可以正常求解。

对四类截断输入，均首先依据式 ((10) 式) 重构 Ω_1 ，再沿用式 ((3) 式)–((6) 式) 的统一模型，并采用图 4 所示的连续分支定界算法求解。

不同代表性输入的计算结果如表 2 所示。

从表 2 可以观察到两个较明显的规律：

首先，三个未截断情形具有相同的最坏定位结果，推荐点仅随第一次位置和示向方向发生相应的平移和旋转，说明前述标准化处理是有效的。

其次，一旦目标圆边界截断第一次目标可行域，推荐第二检测点会随着截断方向和程度发生明显变化。同时，截断区域的存在排除了大量潜在目标位置，使得部分边界案例的最坏定位直径

表 2 不同首次检测情形下的第二检测点求解结果

案例	(S_1, θ_1)	推荐第二检测点 S_2 / m	LB / m	UB / m	Gap / m
中心	$((0, 0), 0^\circ)$	$(843.365, \pm 545.021)$	110.963	110.973	0.010
旋转平移未截断	$((0, 100), 90^\circ)$	$(\pm 545.021, 943.365)$	110.963	110.973	0.010
边界朝内未截断	$((1700, 0), 180^\circ)$	$(856.635, \pm 545.021)$	110.963	110.973	0.010
边界朝外	$((1700, 0), 0^\circ)$	$(1777.802, \pm 65.939)$	4.719	5.208	0.489
边界切向	$((1700, 0), 90^\circ)$	$(1331.175, 421.724)$	28.586	28.942	0.356
边界斜向	$((1700, 0), 45^\circ)$	$(1716.518, 125.959)$	6.525	6.918	0.393
圆外朝内	$((1900, 0), 180^\circ)$	$(937.741, \pm 504.612)$	98.303	98.495	0.192
退化相切单点	$((3300, 0), 180^\circ)$	$(1800, 0)$	0	0	0

反而显著小于基准情况. 若我们将各推荐位置统一转换到以第一次示向方向为横轴的局部坐标系, 可以发现三个未截断案例在局部坐标中重合, 而不同截断情形则产生明显偏移. (图例见附录)

(3) 特殊边界情况

对于特殊边界情况, 模型同样能够正确处理. 当首次检测信息使目标可行域退化为单点时, 可直接将该点作为第二检测位置, 此时定位误差为零; 而当首次检测结果与目标区域及接收约束不相容时, 有 $\Omega_1 = \emptyset$, 模型将直接判定该组观测无可行目标, 不再进行第二检测点优化. 由此说明, 该模型能够同时处理一般情形、退化情形与不相容输入.

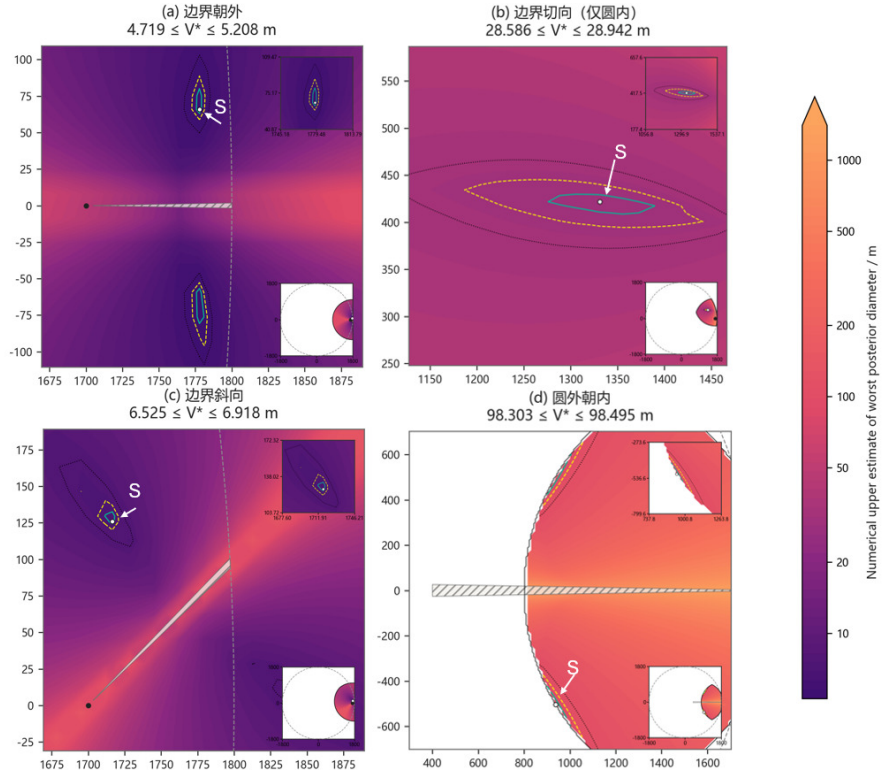


图 6 四类目标圆边界截断情形下的目标可行域、保证接收区域及推荐第二检测点。

白色箭头标示推荐第二检测点 S

6.3 保证二次有效接收约束的影响

前述模型均要求第二检测点位于 C_{sig} 内, 以保证第二次到达后一定能够获得 direction 或 near 有效反馈. 但事实上这一要求并非题目原文中的强制条件, 因此本文进一步以基准情形为例, 研

究全局条件下的结果.

当不再要求 $s \in C_{\text{sig}}$ 时, 第二次还可能出现未接收到信号的情况. 此时 no-signal 本身也提供了一定的信息, 因为它可以排除那些必然能够被当前位置接收到的目标.

对应的后验集合为

$$\Omega_{\text{ns}}(s) = \{g \in \Omega_1 : \|g - s\|_2 > L(g)\}. \quad (17)$$

因此, 取消保证接收限制后的第二检测点性能, 需要同时考虑 near、direction 和 no-signal 三类结果:

$$\tilde{U}(s) = \max \left\{ D(\Omega_{\text{near}}(s)), \sup_z D(\Omega_{\text{dir}}(s, z)), D(\Omega_{\text{ns}}(s)) \right\}. \quad (18)$$

在基准情形下进行相同的连续位置搜索, 得到推荐位置约为

$$S_{2,\text{free}} \approx (970.926, \pm 494.980) \text{ m}, \quad (19)$$

相应的最坏定位直径约满足

$$97.980 \lesssim \tilde{V}^* \lesssim 98.075 \text{ m}. \quad (20)$$

与保证二次接收模型相比, 取消这一约束后可以获得更小的最坏定位直径, 但所得推荐点已经位于原保证接收区域 C_{sig} 之外. 这说明允许 no-signal 后, 机器狗可以利用“未接收到信号”这一结果继续缩小目标范围, 但代价是第二次检测不再保证获得正常示向信息. 若仅追求理论上的

表 3 两种第二检测策略的推荐点及最坏定位结果

策略	推荐第二检测点 S_2 / m	LB / m	UB / m
保证二次有效接收	$(843.365, \pm 545.021)$	110.963	110.973
允许 no-signal	$(970.926, \pm 494.980)$	97.980	98.075

最坏定位区域最小, 可以适当放宽保证接收限制; 若更强调实际检测过程的稳定性和第二次观测的可靠性, 则采用本文的保证二次有效接收模型更加稳妥.

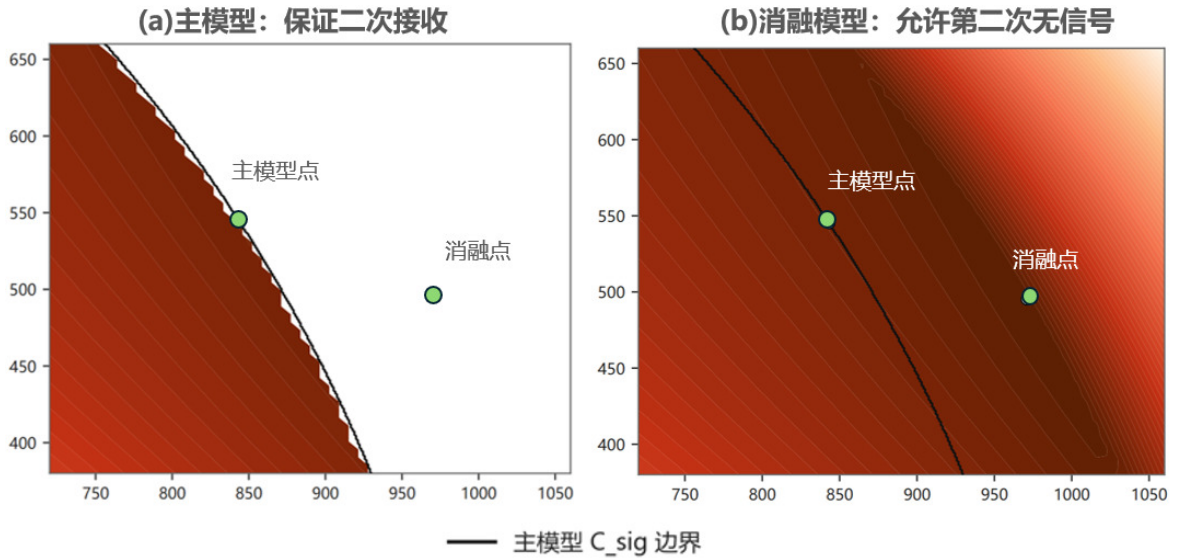


图 7 基准情形下保证二次有效接收与允许 no-signal 两种策略的推荐检测点比较

6.4 模型可靠性与适用范围

为验证前述连续分支定界与连续反馈区间验证并非有限网格近似, 本文记录基准情形的最终分支划分规模. 最终共得到 1536 个连续位置子区域, 其中 750 个由于不满足保证接收条件而被排除, 其余区域均建立了相应的目标函数下界.

同时, 对推荐检测点的所有可能第二次 direction 反馈划分为 108 个连续角度区间, 并分别计算其最坏后验定位区域, 从而避免仅检查有限角度采样点所带来的遗漏.

由此得到基准情形

$$110.963 \leq V^* \leq 110.973 \text{ m},$$

说明推荐第二检测点在本文采用的最坏定位直径指标下具有较高的稳定性.

需要说明的是, 本模型建立在连续 $\pm 1^\circ$ 有界测向误差假设下, 其结论对应题目给出的确定性误差模型. 对于不同的误差分布、接收半径模型或其他任务目标, 应相应调整后验集合和评价函数.

七、问题三：干扰源的自动搜索、定位与清除

7.1 模型建立

7.1.1 时间代价模型

设机器狗总移动距离为 L , 检测次数为 M , 发生频道切换的检测次数为 S , 成功和失败清除次数分别为 C_+ 与 C_- . 根据题目计时规则, 一局任务的总虚拟时间为

$$T = \frac{L}{5} + 5M + S + 5C_+ + 3C_- \quad (21)$$

设该场景实际共有 N 个干扰源, 则评价指标为平均定位清除时间

$$J = \frac{T}{N} \quad (22)$$

在同一场景中 N 固定, 因此最小化平均定位清除时间等价于最小化总任务时间 T . 后续路径规划均以式 ((21) 式) 为时间代价基础.

7.1.2 未知频道的覆盖搜索模型

七站覆盖模型 所有干扰源均位于半径 1800 m 的圆形区域内, 有效接收半径满足 $1000 \leq r \leq 1500$. 因此, 只需使目标区域内任一点距离至少一个扫描站不超过 1000 m, 即可保证相应频道在扫描过程中至少获得一次信号.

本文设置原点和六个外围扫描站. 六个外围站均匀分布在半径 $a = 1150 \text{ m}$ 的圆周附近 5 m 格点:

$$(1150, 0), (575, 995), (-575, 995), (-1150, 0), (-575, -995), (575, -995).$$

为验证覆盖充分性, 将半径 1800 m 的目标圆域外包为正 64 边形, 记其外接圆半径为

$$R' = \frac{1800}{\cos(\pi/64)}.$$

六个外围站形成的正六边形内部, 其最坏最近站距离不超过 $a/\sqrt{3}$.

对于正六边形外部、目标圆域内部的区域, 最坏位置出现在两个相邻外围站夹角的角平分线上, 其最近站距离为

$$\sqrt{R'^2 + a^2 - \sqrt{3}R'a}.$$

考虑外围站坐标向 5 m 格点取整产生的最大偏差 $5/\sqrt{2}$, 得到整个目标区域的覆盖距离上界

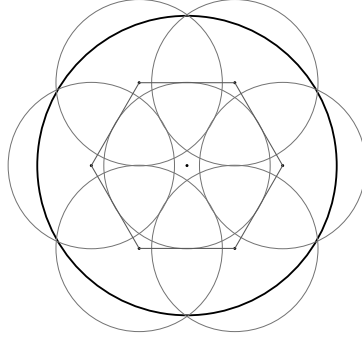


图 8 原点和六外围站组成的七站覆盖结构

$$\bar{r}_{\text{cov}}(a) = \max \left\{ \frac{a}{\sqrt{3}}, \sqrt{R'^2 + a^2 - \sqrt{3}R'a} \right\} + \frac{5}{\sqrt{2}}. \quad (23)$$

代入 $a = 1150 \text{ m}$, 有

$$\bar{r}_{\text{cov}}(1150) = 993.814 \text{ m} < 1000 \text{ m}.$$

因此, 目标区域中的任一干扰源均至少能在一个扫描站进入其最小有效接收半径, 七站方案可以完成未知频道的覆盖搜索.

7.1.3 基于示向约束的集合定位

示向区域更新 设机器狗在位置 s 对某频道进行检测, 获得示向角 θ . 由于测向误差不超过 1° , 对示向结果保留两位小数, 取 $\alpha = 1.005^\circ$ 作为实际定位时的角度半宽.

沿用问题二中楔形可行域 $W(s, \theta, \alpha)$ 的定义, 一次正常示向结果给出的目标位置约束为

$$G \in W(s, \theta, 1.005^\circ) \cap D(s, 1500). \quad (24)$$

设当前目标可能位置的凸外包为 P , 则每次正常示向后更新为

$$P \leftarrow P \cap W(s, \theta, \alpha) \cap D(s, 1500).$$

若检测结果为 **near**, 则直接更新

$$P \leftarrow P \cap D(s, 5).$$

正示向与 **near** 反馈用于更新保证性的凸位置外包 P . 对于无信号结果, 由于目标有效接收半径不小于 1000 m , 可知目标不位于检测点的 1000 m 邻域中; 清除失败则说明目标距离当前点超过 20 m . 这两类负反馈单独记录, 用于频道排除和候选动作评价.

可靠清除区域 设当前位置外包 P 的顶点为 $v_1, \dots, v_m$. 为避免直接在 20 m 的物理边界上执行清除, 取 $R_s = 19.7 \text{ m}$.

定义可靠清除区域

$$C = \bigcap_{i=1}^m D(v_i, R_s). \quad (25)$$

若 $y \in C$, 对于 P 中任意位置

$$x = \sum_i \lambda_i v_i, \quad \lambda_i \geq 0, \quad \sum_i \lambda_i = 1,$$

由欧氏范数的凸性可得

$$\|y - x\| \leq \sum_i \lambda_i \|y - v_i\| \leq 19.7 < 20. \quad (26)$$

因此, 当 C 非空时, 在其中任一点执行清除均可成功. 若只考虑当前目标, 可取

$$y^{\text{near}} = \arg \min_{y \in C} \|y - p_t\| \quad (27)$$

作为候选服务点; 多目标情况下的最终服务位置还需结合后继任务和连续清除块的整体路径代价确定.

7.1.4 多目标滚动规划模型

仅仅完成单个目标的定位并不是本问的最终目的. 若机器狗每发现一个目标便立即定位、清除, 再返回扫描路线, 会产生大量往返. 因此需要同时考虑未知频道扫描和已发现目标服务的平衡. 由于新的示向结果会持续改变目标位置和待服务任务集合, 查阅文献后, 本文采用动态路径规划中的滚动重优化思想^[11]: 每次仅执行当前方案的首个任务, 并在获得新反馈后重新生成任务和访问顺序.

服务任务构造 对于每个已发现目标, 根据当前定位结果生成三类候选任务:

- (1) **补充检测**: 移动到新的观测位置, 进一步缩小目标位置范围;
- (2) **试探清除**: 当目标已经较集中但尚不能保证成功时, 直接尝试清除;
- (3) **可靠清除**: 当式 ((25) 式) 非空时, 进入可靠清除区域.

尚未完成的扫描站同样作为待访问任务. 这样, 原问题转化为不断变化的多任务访问问题.

设任务 i 的入口位置为 p_i , 执行时间为 τ_i , 任务 i 完成后到任务 j 的预计移动时间为 d_{ij} . 对于访问序列 $\sigma = (\sigma_1, \sigma_2, \dots, \sigma_k)$,

其预计总代价为

$$\hat{T}(\sigma) = \frac{\|p_t - p_{\sigma_1}\|}{5} + \sum_{j=1}^k \tau_{\sigma_j} + \sum_{j=1}^{k-1} d_{\sigma_j, \sigma_{j+1}}. \quad (28)$$

7.1.5 访问次序求解

每个目标对应一个任务组 j , 其候选服务模式集合记为 M_j . 设 U 为尚未访问的任务组集合, i 为上一服务模式, q 为当前接收频道, 定义

$$F(U, i, q) = \min_{\substack{j \in U \\ m \in M_j}} [d_{im} + \tau_m(q) + F(U \setminus \{j\}, m, q'_m(q))], \quad (29)$$

其中 $F(\emptyset, i, q) = 0$. d_{im} 为模式之间的移动时间, $\tau_m(q)$ 包含模式内检测、清除及切频成本, $q'_m(q)$ 为执行后的接收频道; 初始状态以机器狗当前位置代替上一服务模式. 查阅到 Held 和 Karp 1962 用动态规划解决 sequencing/TSP 类访问次序问题的经典工作和 Ow 和 Morton 关于 beam search 在大规模决策树搜索中的使用的讨论, 我们决定: 任务组数不超过 7 时, 采用 Held-Karp 型子集动态规划^[12] 对服务顺序进行精确搜索; 当任务组数超过 7 时, 为限制组合状态空间, 采用固定宽度束搜索 (beam search)^[13], 本文取束宽为 64, 并利用剩余服务成本和最小生成树松弛下界对部分路径排序. 每次只执行当前规划中的第一个任务, 获得新反馈后重新生成任务并规划.

7.1.6 有限蛇形后备模型

滚动定位可以显著减少平均耗时, 但仍需保证在定位效果不理想时能够完成清除.

设第一次示向位置为 s , 并以示向中心方向为局部 x 轴. 目标至该位置距离不超过 1500 m, 角度误差不超过 1.005° , 因此有

$$0 \leq x \leq 1500, \quad |y| < 1500 \tan(1.005^\circ) < 27 \text{ m}.$$

由此, 在局部坐标系中设置 $x = 0, 20, \dots, 1500, y \in \{-40, -20, 0, 20, 40\}$, 共得到 $76 \times 5 = 380$ 个候选清除点, 并按蛇形顺序访问.

20 m 网格带来的最大距离误差为 $20/\sqrt{2}$, 坐标量化到 5 m 格点后再增加至多 $5/\sqrt{2}$, 即

$$e_{\max} \leq \frac{20}{\sqrt{2}} + \frac{5}{\sqrt{2}} = 17.678 \text{ m} < 20 \text{ m}. \quad (30)$$

所以这 380 个候选点中必有至少一个位于真实源 20 m 清除范围内, 蛇形搜索能够在有限次尝试内完成清除.

实际运行中, 每次普通决策前均检查“拟执行动作的最坏成本+剩余后备成本上界”是否仍满足虚拟时间与动作额度约束; 若不满足, 则停止普通滚动规划, 完成剩余覆盖扫描, 并对已发现而未清除的目标执行上述有限后备.

(1) 利用固定扫描站完成未知频道的覆盖搜索;

(2) 对已发现目标采用有界误差集合进行连续定位, 并判断是否能够可靠清除;

(3) 将扫描、补充测量和清除统一为待访问任务, 滚动优化机器狗的下一步行动.

由此形成“覆盖搜索-集合定位-滚动规划-可靠清除”的总体策略; 滚动过程中同步进行预算监督, 当继续普通决策会使剩余资源不足以完成后备方案时, 转入有限蛇形后备. 总体流程如图 9 所示.

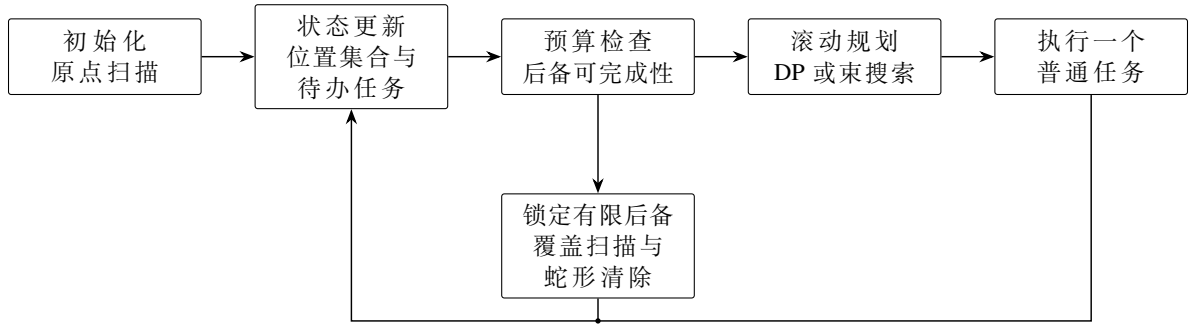


图 9 问题三自动搜索、定位与清除流程

7.1.7 可靠清除块的路径优化模型

当多个目标均已形成可靠清除区域时, 若分别选择距当前位置最近的清除点, 可能导致整体路线出现较大折返. 因此, 在目标访问顺序确定后, 对连续清除块进一步优化.

设清除块入口为 A , 出口为 B , 第 i 个目标的可靠清除区域为 C_i , 求解

$$f^* = \min_{y_i \in C_i} \left[\|A - y_1\| + \sum_{i=1}^{m-1} \|y_i - y_{i+1}\| + \|y_m - B\| \right]. \quad (31)$$

由于每个 C_i 均为圆盘交形成的凸集, 且目标函数为欧氏距离之和, 因此式 ((31) 式) 为凸优化问题. 采用平滑目标

$$f_\varepsilon = \sum_{e=1}^E \sqrt{\|d_e\|^2 + \varepsilon^2}, \quad \varepsilon = 10^{-5},$$

其中 E 为清除块路径边数. 于是有

$$f_\varepsilon - E\varepsilon \leq f \leq f_\varepsilon. \quad (32)$$

在当前可行点 y 处, 记

$$g_i = \nabla_{y_i} f_\varepsilon(y), \quad \ell_i \leq \min_{z \in C_i} g_i^\top z.$$

由凸函数的一阶支撑不等式可得

$$L = f_\varepsilon(y) - E\varepsilon + \sum_i (\ell_i - g_i^\top y_i) \leq f^* \leq U = f(y^{\text{feas}}). \quad (33)$$

因此 $U-L$ 可直接评价固定顺序连续子问题的求解精度。

优化坐标量化后重新核验可靠清除条件，并按整数微秒重算清除块成本；只有优化后成本不高于原可行方案时才采用新方案。

7.2 模型求解

7.2.1 求解算法

综合上述模型，问题三的求解过程如下。

问题三自动搜索定位清除算法

Step 1: 初始化与覆盖任务生成. 从原点开始扫描，并将其余六个扫描站作为待完成覆盖任务加入滚动规划；扫描过程中发现目标后，立即加入相应的定位与清除任务。

Step 2: 集合定位. 利用新的示向、近距离和负反馈结果更新目标位置范围，并判断可靠清除区域是否形成。

Step 3: 任务生成. 根据当前状态生成剩余扫描、补充检测、试探清除和可靠清除任务。

Step 4: 滚动规划. 联合选择下一任务及其服务模式；任务组数不超过 7 时采用完整子集动态规划；超过 7 时采用宽度 64 的束搜索，并利用剩余服务成本和最小生成树松弛下界对部分路径排序。

Step 5: 执行与更新. 仅执行规划中的第一个任务，根据实际反馈重新定位和规划。

Step 6: 预算监督与有限后备. 每次普通决策前检查执行该动作后是否仍有足够的虚拟时间和动作额度完成后备；若不足，则停止普通滚动决策，完成剩余覆盖扫描并执行有限扇区蛇形清除。

Step 7: 终止. 全部已发现目标完成清除，且所有其他频道均完成覆盖排除后结束任务。

7.2.2 算法完成性分析

七站覆盖保证目标区域内任一点距离某一扫描站小于 1000 m，因此任一真实存在的频道均可在有限扫描中被发现。对于已经发现的目标，若可靠清除区域非空，则由式 ((26) 式) 可直接完成清除；否则由式 ((30) 式) 可知，380 个蛇形候选点中至少存在一个成功清除点。

滚动规划在每次普通动作前检查剩余虚拟时间和动作额度是否仍足以完成上述后备过程；余量不足时后备。因此覆盖搜索、可靠清除、有限蛇形后备和预算监督共同构成有限完成机制。

上述结论建立在题设物理约束、位置外包与可靠判定正确、接口正常接受动作以及后备预算充足的条件下。

7.3 模型结果与分析

7.3.1 实验设计

为考察算法在不同干扰源数量下的表现，令 $N = 10, 11, \dots, 16$,

每种目标数量固定生成 20 个测试场景，共进行 140 场演练。

主评价量采用各目标数分组均值的等权平均。设第 n 组均值和标准差分别为 $\bar{X}_n, s_n$ ，则

$$\bar{X} = \frac{1}{7} \sum_{n=10}^{16} \bar{X}_n, \quad (34)$$

相应方差估计为

$$\widehat{\text{Var}}(\bar{X}) = \frac{1}{49} \sum_{n=10}^{16} \frac{s_n^2}{20}. \quad (35)$$

7.3.2 总体结果

140 场共包含 1820 个干扰源，全部完成清除。平均定位清除时间为 235.80 s/目标，中位数为 231.22 s/目标，P90 为 282.48 s/目标。均值略高于中位数，表明少数较慢场景形成了右侧尾部。

表 4 问题三 140 场演练总体结果

场数	目标数	全清场数	均值	标准差	中位数	P90	95% 区间
140	1820	140	235.80	33.34	231.22	282.48	[232.51, 239.10]

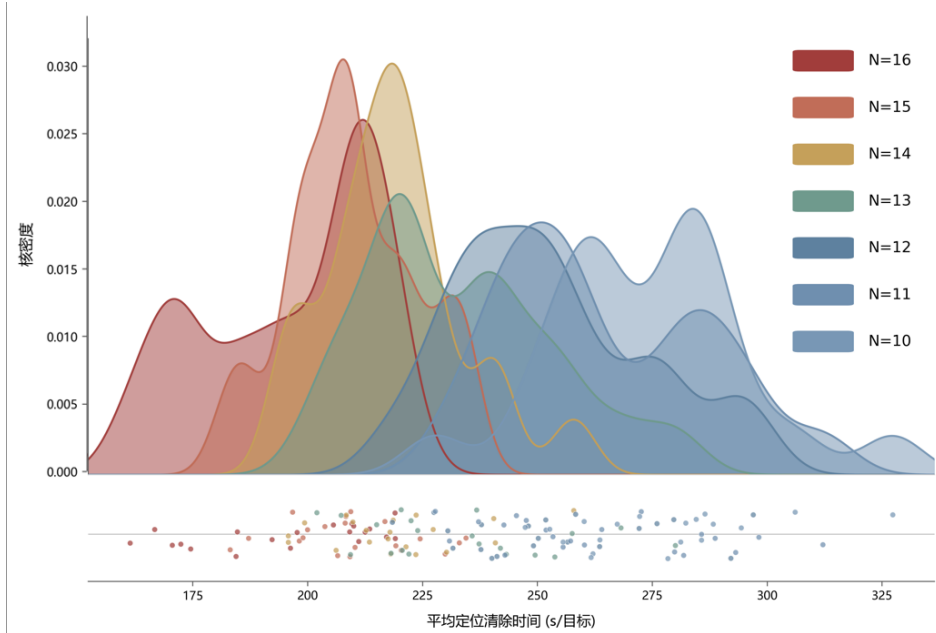


图 10 问题三 140 场平均定位清除时间的经验累积分布

7.3.3 不同目标数下的表现

随着干扰源数量由 10 增加到 16，平均单目标时间由 274.23 s 下降到 196.36 s。这一趋势可能与扫描路线等固定成本被更多目标分摊、以及空频道数量减少有关。

7.3.4 时间组成

移动时间占总时间的 78.02%，是最主要的耗时来源。最慢场景的平均时间为 327.31 s/目标，其中相对于同目标数组均值增加的 53.07 s/目标中，有 47.91 s 来自额外移动。因此，进一步提高效率的关键是减少扫描和目标服务之间的长距离折返。

7.4 正式测试结果

三次正式测试均完成全部已发现干扰源的搜索与清除，结果如表表 7 所示。

表 5 不同目标数下的平均定位清除时间

N	场数	均值/(s/目标)	标准差/(s/目标)	P90/(s/目标)
10	20	274.23	22.14	295.48
11	20	265.03	22.66	293.64
12	20	252.94	21.58	280.89
13	20	233.25	20.94	258.58
14	20	218.93	15.55	240.51
15	20	209.88	14.17	230.20
16	20	196.36	19.13	217.33

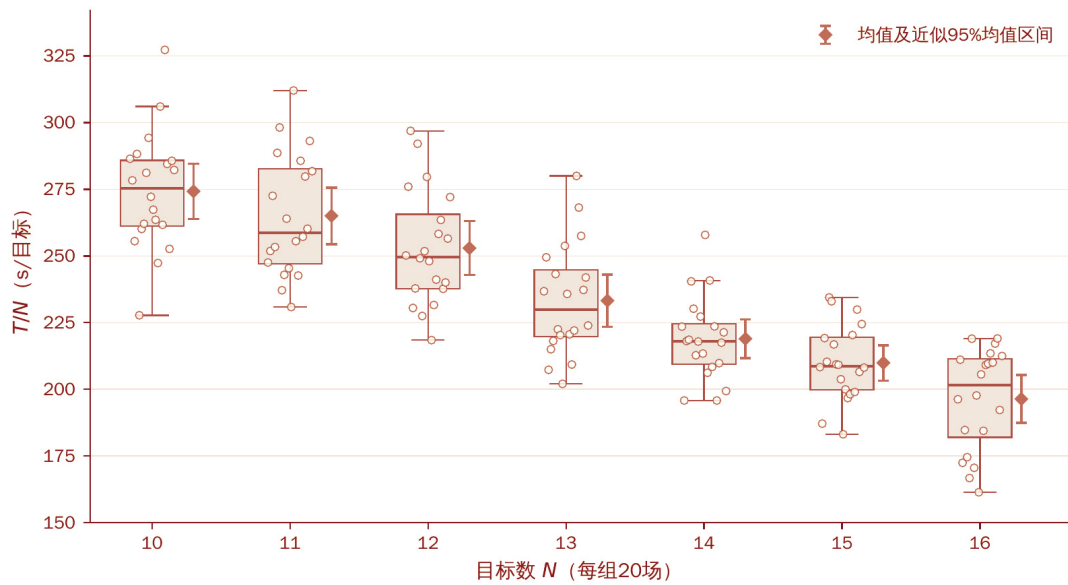


图 11 不同干扰源数量下的平均定位清除时间分布

表 6 问题三虚拟时间组成

时间分量	平均耗时/(s/目标)	占比/%
移动	183.59	78.02
检测	38.63	16.21
切频	7.31	3.07
成功清除	5.00	2.16
失败清除	1.27	0.55

三次测试共清除 40 个干扰源，平均定位清除时间为

254.77 s/目标,

三次结果的样本标准差为

8.05 s/目标.

各次测试结果较为集中，与前述 140 场演练所得时间尺度一致.

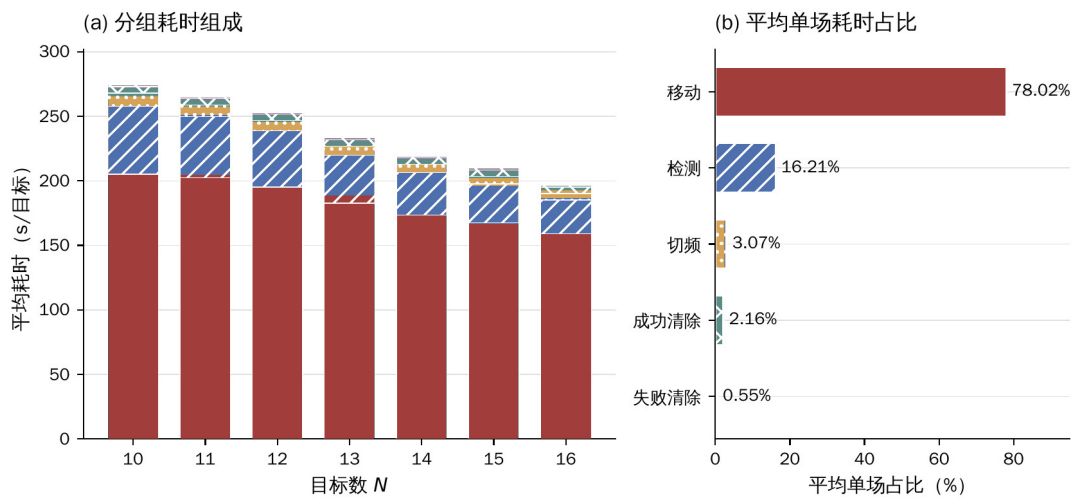


图 12 问题三 140 场演练的虚拟时间组成

表 7 问题三正式测试结果

测试	测试案例编码	清除干扰源个数	平均定位清除时间/(s/目标)	程序运行时间/s
1	CBE2-22SN-GJGY-U52F	14	264.055	25.000
2	VW6S-RUB8-9VE5-7TB3	12	250.598	32.734
3	BUZW-PQPG-H6Z3-EDNM	14	249.672	32.969

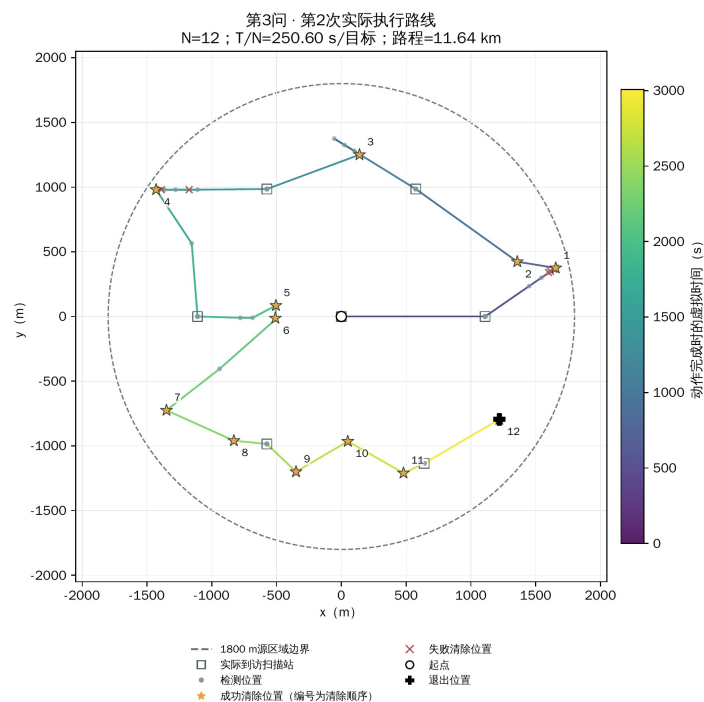


图 13 问题三第 2 次正式测试的实际运行路线

图 13 给出了第 2 次测试的实际运动轨迹。该次共清除 12 个干扰源，机器狗总移动距离为 11.636 km，平均定位清除时间为 250.60 s/目标。可以看出，覆盖扫描与目标清除并非两个彼此独立的阶段，而是在运行过程中交替进行；滚动规划能够根据新获得的定位结果不断调整后续访问顺序。

三次测试的平均定位清除时间分别为 264.05 s、250.60 s 和 249.67 s/目标。本次三场结果较为集中，并与 140 场演练所得主要时间尺度一致。

7.5 模型评价

本文模型将区域覆盖、测向定位与动态路径规划相结合，能够在目标位置未知的情况下完成搜索、定位和清除全过程。模型充分利用了题目给出的测向误差和清除距离约束，整体建模过程与实际任务机制较为一致，具有较好的合理性。

从实验结果看，模型在不同干扰源数量和不同随机场景下均能完成清除任务，且正式测试结果与演练结果基本一致，说明模型具有较好的稳定性和鲁棒性。与此同时，模型采用滚动更新策略，根据当前已获得的信息实时调整后续搜索和清除路径，因此相比固定路线方案具有更强的适应性。

模型的主要不足在于，当前路径规划仍以局部决策为主，对未来目标分布和后续移动代价考虑有限。当干扰源数量进一步增加时，可能出现局部最优而整体移动距离偏大的情况。因此，该

模型适用于目标位置未知、需要保证完整搜索与可靠清除的场景。后续可进一步引入全局路径优化和预测性规划，在保证清除成功率的同时进一步降低总移动距离和任务完成时间。

八、问题四：定向干扰源下的自动搜索、定位与清除

8.1 模型建立

8.1.1 位置-方向联合覆盖模型

全朝向可见性的凸包判据 给定一组扫描点 S ，对固定源位置 g 定义其最小接收半径内的扫描点集合

$$S_g = \{s \in S : \|s - g\| \leq 1000\}. \quad (36)$$

命题 8.1 (全朝向可见性的凸包判据) 对任意固定位置 g ，有

$$\forall u \in \mathbb{S}^1, \exists s \in S_g : (s - g)^\top u \geq 0 \iff g \in \text{conv}(S_g). \quad (37)$$

该判据将未知发射方向下的可见性条件转化为扫描点的凸包包含条件，从而可以直接用于连续区域的覆盖设计。该等价关系可由凸集分离定理得到^[14]，从而将未知发射方向下的可见性条件转化为扫描点的凸包包含条件。具体证明见附录 3.1。

三角单元覆盖充分性条件 为在连续圆域上验证式 ((37) 式)，将目标区域划分为三角形单元 T 。若存在扫描点集合 W_T 满足

$$T \subseteq \text{conv}(W_T), \quad \max_{s \in W_T} \max_{v \in V(T)} \|s - v\| < 1000, \quad (38)$$

则单元内任意位置均满足式 ((37) 式)，从而对任意发射朝向均至少存在一个可见扫描点。该条件的推导及连续区域覆盖证明见附录 3.1。

21 点位置-方向覆盖布局 根据上述判据，本文采用由原点、半径 995 m 的 8 个等分内圈点和 12 个外圈点组成的 21 点基础布局。外圈正十二边形的设计内切圆半径取 1801 m，因此外圈顶点半径为

$$r_{\text{out}} = \frac{1801}{\cos 15^\circ} \approx 1864.53 \text{ m},$$

从而包围半径 1800 m 的目标区域。

对目标区域进行自适应三角剖分，并对每个叶单元检查式 ((38) 式) 中的凸包包含关系和距离条件，以验证 21 点布局在连续目标区域上的位置-方向覆盖充分性。

基础布局及三角剖分验证结果如图图 14 所示。

完整覆盖执行后，若某频道始终没有正反馈，即可判定该频道不存在干扰源；若已经发现 16 个不同频道的真实源，则由题目给定的数量上界直接排除其余未知频道。21 点布局的连续覆盖证明及旋转后覆盖性质见附录 3.1。

8.1.2 集合定位与可靠清除

问题四沿用问题三的示向楔形外包、near 更新和清除规则。与问题三不同的是，定向源返回 no_signal 时，不能直接按距离排除位置区域。本文将该类负反馈单独记录，用于更新位置-方向覆盖状态和候选动作评价，而可靠清除仍基于保守位置外包判断。

8.1.3 覆盖与目标服务的滚动联合规划模型

第四问同时存在尚未完成的位置-方向覆盖扫描和已发现目标的继续测量、清除任务。若先完成全部扫描再逐一一定位，会产生较大往返，因此程序在每次反馈后重新构造两类任务并统一规划。

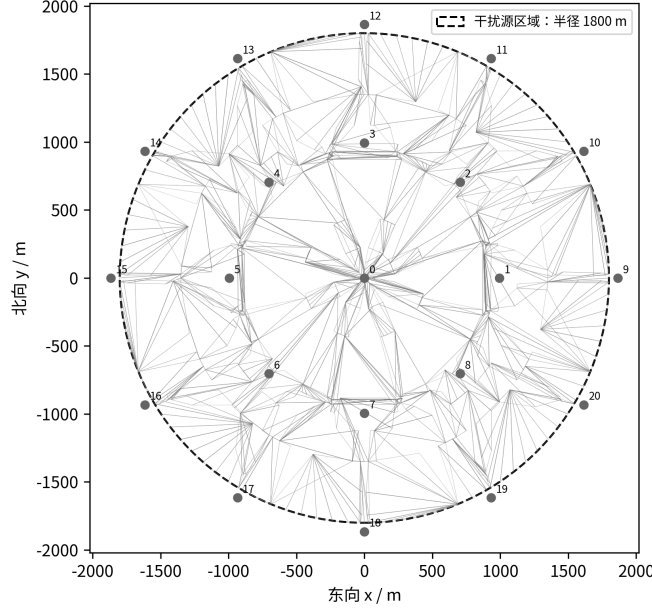


图 14 问题四 21 点位置-方向联合覆盖布局及三角剖分验证

剩余覆盖路径 MILP 设 y_i 表示是否访问扫描站 i , x_{ij} 表示是否使用路径边 (i, j) , z_k 表示是否启用第 k 个凸包联合覆盖条件. 当前剩余覆盖路径模型写为

$$\min \left[\frac{1}{5} \sum_{(i,j) \in E} d_{ij} x_{ij} + \sum_i \tau_i y_i \right]. \quad (39)$$

该问题本质上属于带覆盖要求的路径访问组合优化, 本文参考经典车辆路径规划的整数规划建模框架^[15] 建立剩余覆盖路径 MILP. 对每个尚未满足的覆盖要求 ℓ , 记 D_ℓ 为能够直接覆盖它的扫描站集合, K_ℓ 为能够联合证明其覆盖的见证集合, 则要求

$$\sum_{i \in D_\ell} y_i + \sum_{k \in K_\ell} z_k \geq 1. \quad (40)$$

历史观测已经满足的覆盖要求在重新规划时直接删除. 访问变量与联合覆盖条件的联动、开放路径度约束和连通割见附录 3.2.

服务方式-访问顺序联合 MILP 对于已发现目标组 g , 构造有限候选模式集合 M_g , 包括继续测量、条件清除和可靠清除等方式. 令 y_m 表示是否选择模式 m , 则每个当前目标组恰选一个服务模式:

$$\sum_{m \in M_g} y_m = 1, \quad \forall g. \quad (41)$$

随后在所选模式之间建立开放路径 MILP, 同时确定服务方式和访问顺序; 其中路径连通采用流守恒约束, 并利用经典 MTZ 约束消除子环^[16]. 未来接收频道依赖服务顺序, 因此规划阶段对未知未来频道采用

$$\tau_m = \min_q \tau_m(q) \quad (42)$$

构造切频成本松弛; 得到候选路线后, 再从当前真实频道出发逐项恢复频道状态并重算可执行成本.

设当前冻结候选目录上的最优预测成本为 $J_t^{*, \text{cat}}$. 由重算后的可执行路线得到上界 U_t , 由有效 MILP 松弛得到下界 L_t , 于是

$$L_t \leq J_t^{*, \text{cat}} \leq U_t. \quad (43)$$

每轮仅执行当前方案的第一个任务, 获得新反馈后重新生成覆盖要求和服务模式, 再次求解.

8.1.4 有限几何后备模型

当单个频道累计达到 8 次局部测量或 4 次失败清除而仍未完成时，算法转入 25 m 格点后备。由于 25 m 方格内任意位置到最近格点的距离不超过

$$\frac{25}{\sqrt{2}} = 17.68 \text{ m} < 20 \text{ m}, \quad (44)$$

在保守位置外包上有限枚举不重复格点即可形成清除后备。其有限性与条件完备性证明见附录 3.3。总体流程如图 15 所示。

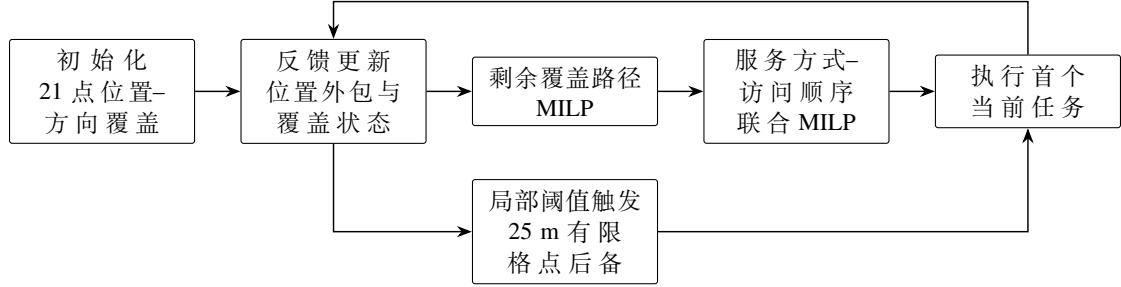


图 15 问题四位置-方向覆盖与联合 MILP 滚动流程

8.2 模型求解

综合上述模型，问题四的执行过程如下。

问题四位置-方向联合搜索清除算法

Step 1: 建立认证覆盖。 原点完成初始扫描后，构造 21 点位置-方向覆盖布局；若根据已发现目标选择整体旋转角，则对旋转并量化后的布局重新检查目标圆包含、凸包包含及距离条件，验证通过后更新覆盖布局。

Step 2: 更新覆盖与目标状态。 根据新反馈同步更新剩余位置-方向覆盖要求以及已发现目标的位置外包，并生成覆盖扫描和目标服务任务。

Step 3: 联合滚动规划。 先求解剩余覆盖路径 MILP，再求解服务方式-访问顺序联合 MILP；对所得候选路线从当前真实频道出发恢复频道状态并重算可执行成本。每轮仅执行首个任务，获得新反馈后重新生成模型并求解。

Step 4: 局部后备与终止。 若单个目标达到 8 次局部测量或 4 次失败清除仍未完成，则转入 25 m 有限格点后备，全部源清除且其余频道完成覆盖排除后结束。

8.2.1 算法条件完备性

在位置外包保持保守、覆盖任务与已发现目标获得公平服务、后备点不重复且运行资源充足的条件下，21 点位置-方向覆盖保证真实源能够被发现，可靠清除与 25 m 有限后备保证已发现目标能够被清除。因此算法具有条件有限完备性，详细证明见附录 3.3。

8.3 模型结果与分析

8.3.1 实验设计结果

一次性生成 100 个随机场景并全部运行。场景中 N 在 10–16 之间均匀取整，源位置在半径 1800 m 圆盘内按面积均匀生成，有效接收半径在 1000–1500 m 间均匀生成；定向源个数在 1 到 $N - 1$ 之间随机，其发射朝向均匀生成。

100 场均完成全部干扰源清除。总体结果如表 8 所示。

单位目标耗时最小为 268.35 s/源，最大为 629.25 s/源。批量实验采用 4 进程并行，单场程序运行时间平均 119.20 s，中位数 100.69 s，最大 580.35 s；程序运行时间用于描述计算开销，不计入题目定义的虚拟时间。

表 8 问题四 100 场随机仿真实验总体结果

场数	全清场数	均值	标准差	中位数	P90	P95	平均路程/km
100	100	449.58	86.23	444.90	562.57	595.94	20.53

8.3.2 不同源数量下的表现

表 9 问题四不同干扰源数量下的分层统计

N	场数	均值/(s/源)	组内标准差/(s/源)	中位数/(s/源)
10	12	590.18	22.99	594.27
11	21	520.59	20.30	520.04
12	14	476.23	20.44	471.54
13	15	432.68	12.06	430.98
14	12	399.64	18.31	397.36
15	13	374.28	18.90	371.79
16	13	317.26	38.80	298.92

随着 N 增加，平均单目标时间明显下降。这一趋势可能来自固定覆盖成本被更多目标分摊、空频道数量减少以及源数上界带来的提前排除。日志中 13 个 $N = 16$ 场景有 12 场在发现 16 个不同频道后触发数量上界排除，平均直接排除 3.69 个剩余频道。按 N 分组的组间平方和占本批 T/N 总样本平方和的 93.54%，因此总体标准差主要反映不同 N 场景的混合。

图图 16 同时给出了源数量与实际路程的关系。

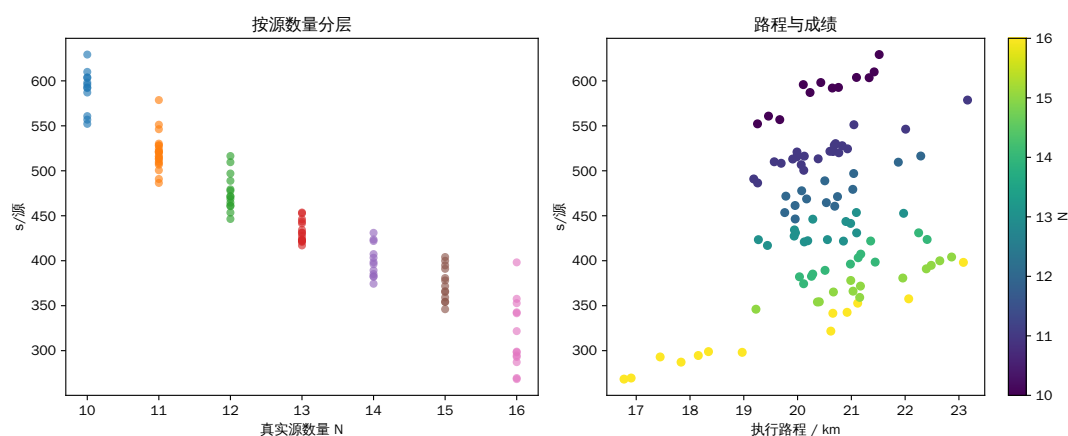


图 16 问题四 100 场随机仿真实验中源数量、单位目标耗时与执行路程

8.3.3 时间组成

100 场累计虚拟时间中，移动、检测、换频和清除分别占 73.11%、21.43%、4.04% 和 1.42%。其中移动仍是最主要成本，检测和换频合计达到 25.47%，反映了定向源需要在多个位置-频道组合上完成搜索和确认。

100 场中末次成功清除到最终退出的平均排查时间为 343.73 s，约占平均总虚拟时间的 6.12%。因此尾段不存在性确认值得继续压缩，主要优化对象仍是移动与覆盖。

8.3.4 正式测试结果

三次 Windows 正式测试结果如表表 10 所示。

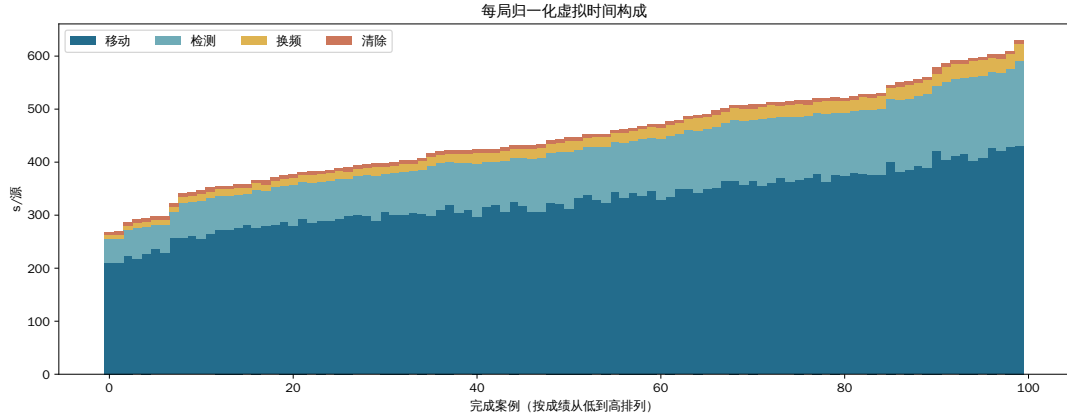


图 17 问题四 100 场随机仿真实验的单位目标时间组成

表 10 问题四正式测试结果

测试	测试案例编码	清除干扰源个数	平均定位清除时间/(s/源)	程序运行时间/s
1	CBE2-22SN-GJGY-U52F	10	574.540	30.594
2	VW6S-RUB8-9VE5-7TB3	12	480.024	83.016
3	BUZW-PQPG-H6Z3-EDNM	11	517.248	38.328

三次测试共清除 33 个干扰源，平均定位清除时间分别为 574.540、480.024 和 517.248 s/源，算术均值为 523.937 s/源。本次三场结果较为集中，并落在 100 场随机仿真实验覆盖的时间范围内。

第 3 次测试的单位目标耗时接近三次中位数，图图 21 给出其实际运行路线。可以看到，扫描站访问与目标定位、清除在同一条路线中交替进行，符合前述滚动联合规划的执行方式。

三次正式测试的时间与路程汇总见附录 3.4。

8.4 模型评价

问题四的核心在于定向干扰源存在发射盲区，使问题三仅依赖距离的覆盖方式不再适用。本文通过位置-方向联合覆盖解决未知发射方向下的源发现问题，并利用滚动规划协调剩余扫描与已发现目标的定位、清除任务。

该模型适用于干扰源类型未知或可能包含定向源的情形；若已知全部干扰源均为全向源，则直接采用问题三的七站覆盖策略即可，无需增加位置-方向覆盖带来的扫描成本。对于问题四模型本身，当前主要效率瓶颈仍来自移动，因此后续改进应优先考虑减少覆盖路径和目标服务之间的折返。

联合 MILP 能够统一处理覆盖任务和目标服务，但会增加在线计算量。因此，在移动代价较高、任务之间耦合明显时采用联合优化更有意义；若计算资源有限，则可保留认证覆盖和基本滚动服务策略，以降低求解复杂度。

九、误差分析

9.1 第三问误差与敏感性分析

9.1.1 扫描站半径的影响

由式 ((23) 式)，外围扫描站半径 a 对应的覆盖距离上界为

$$\bar{r}_{\text{cov}}(a) = \max \left\{ \frac{a}{\sqrt{3}}, \sqrt{R'^2 + a^2 - \sqrt{3}R'a} \right\} + \frac{5}{\sqrt{2}}. \quad (45)$$

部分结果如表表 11 所示。

表 11 外围扫描站半径的敏感性分析

a/m	1130	1135	1140	1150	1160
$\bar{r}_{\text{cov}}/\text{m}$	1002.274	1000.128	998.003	993.814	989.708

由 $\bar{r}_{\text{cov}}(a) < 1000$ 得到临界点约为 1135.30056 m 和 1725.92708 m. 取保守充分区间

$$1135.31 \leq a \leq 1725.92.$$

当前 $a = 1150$ m 位于有效区间内部, 且 $\bar{r}_{\text{cov}}(1150) = 993.8135$ m, 保留约 6.19 m 的覆盖裕量.

9.1.2 蛇形网格步长的影响

若蛇形遍历的网格步长为 h , 则考虑 5 m 坐标量化后的最坏误差为

$$e(h) \leq \frac{h}{\sqrt{2}} + \frac{5}{\sqrt{2}}. \quad (46)$$

由 $e(h) < 20$ 得

$$h < 23.284 \text{ m}.$$

本文取 $h = 20$ m, 对应最坏误差 17.678 m, 因此仍有 2.322 m 的清除距离余量.

9.1.3 连续优化误差

140 场实验共得到 285 个连续清除任务段优化结果, 其中 209 个满足 $U - L \leq 0.002$ m, 占 73.33%; 其余 76 个保留对应的可行上界与支撑下界. 上下界间隙中位数为 7.82×10^{-4} m, P90 为 1.03×10^{-2} m.

9.1.4 运行时间下界

为了衡量实际结果所处的时间尺度, 任务完成后利用真实目标位置构造理想化清除路线. 设目标位置为 G_i . 从原点到第一个 20 m 清除圆盘的最短距离为

$$d_{0i} = \max\{0, \|G_i\| - 20\},$$

两个目标清除圆盘之间的最短距离为

$$d_{ij} = \max\{0, \|G_i - G_j\| - 40\}.$$

利用上述距离作为边权, 采用 Held-Karp 算法^[12] 求原点出发、终点自由的最短开放路径 L_{HK} , 得到时间下界

$$T_{\text{LB}} = \frac{L_{\text{HK}}}{5} + 5N. \quad (47)$$

140 场的平均下界为 132.67 s/目标, 而实际算法平均为 235.80 s/目标. 二者的差距综合反映了未知源搜索、主动定位、频道切换、在线决策以及圆盘边距离松弛等因素带来的附加代价; 结合时间组成结果, 移动仍是实际算法中占比最高的时间分量.

9.2 第四问误差与敏感性分析

9.2.1 位置-方向覆盖的几何误差

已有定向传感覆盖研究表明, 方向性条件下的覆盖设计需要同时考虑空间位置与方向状态^[17]. 针对本题未知发射朝向的特点, 本文进一步构造位置-方向联合覆盖判据. 21 点布局采用连续三角单元逐一验证位置-方向覆盖条件. 在最小接收半径取 1000 m 时, 对应的距离裕量为

$$\Delta_r = 1000 - d_{\text{max}} \approx 0.0131 \text{ m}.$$

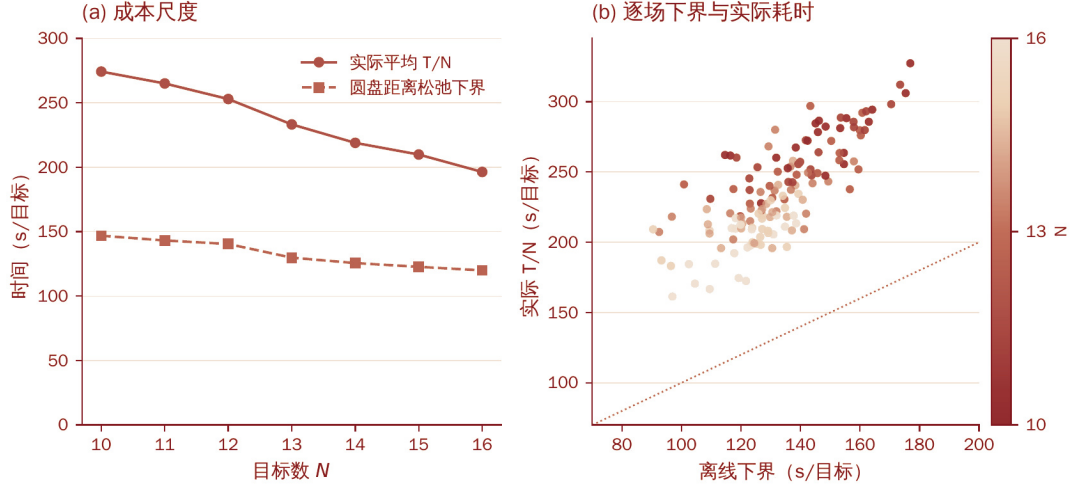


图 18 问题三离线时间下界与实际耗时比较

另一方面，外圈正十二边形的设计内切圆半径为 1801 m，相对于半径 1800 m 的干扰源允许区域保留 1m 的几何包含余量。

由于位置-方向覆盖中的距离裕量较小，本文在扫描布局发生整体旋转并进行坐标量化后，重新检验目标圆包含、凸包包含和距离条件，只有重新验证通过后才采用新的扫描布局。因此坐标量化误差不会直接沿用理想连续布局的覆盖结论。

9.2.2 25 米格点清除网格步长的影响

设正方形网格步长为 h ，则平面任一点到最近格点的最大距离为

$$e(h) = \frac{h}{\sqrt{2}}. \quad (48)$$

为保证至少存在一个格点落入 20 m 清除范围，需要 $\frac{h}{\sqrt{2}} < 20$ ，即 $h < 20\sqrt{2} \approx 28.284$ m。

本文取 $h = 25$ m，此时 $e(25) = \frac{25}{\sqrt{2}} \approx 17.678$ m，因此相对 20 m 清除阈值仍保留约 $20 - 17.678 = 2.322$ m 的距离余量。由此可见，25 m 步长并非取在临界位置附近。

9.2.3 运行时间下界

为了衡量第四问实际结果所处的时间尺度，任务完成后利用真实目标位置构造理想化路线下界。沿用第三问的 20 m 清除圆松弛，定义

$$\underline{d}_{0i} = \max\{0, \|G_i\| - 20\},$$

以及

$$\underline{d}_{ij} = \max\{0, \|G_i - G_j\| - 40\}.$$

在上述距离图上求原点出发、终点自由的最短开放路径，并加入每个目标至少一次成功清除所需的固定时间，得到仅考虑清除所需移动的乐观下界。

100 场实验中，目标中心开放最短路参考时间平均为 1869.99 s，采用 20 m 清除圆距离松弛后得到的物理时间下界平均为 1771.45 s，实际虚拟总时间平均为 5615.69 s。实际耗时与理想化下界之间的差距综合包含位置-方向覆盖搜索、主动定位、频道切换、在线滚动规划以及距离松弛本身造成的影响。结合时间组成结果，移动时间平均占 73.11%，仍是第四问最主要的时间分量。

十、模型的评价及推广

10.1 模型的优点

- 1、本文将测向误差转化为连续几何可行域，并结合最坏情形优化进行定位与检测点选择，使模型具有较强的数学严谨性和可靠性。
- 2、在搜索与清除过程中，根据实时反馈不断更新目标可行区域并动态调整后续路径，能够较好地适应目标位置和数量未知的情况。
- 3、通过误差分析、敏感性分析和多次随机仿真实验对模型进行验证，所得结果具有较好的稳定性与可信度。

10.2 模型的缺点

- 1、模型忽略了遮挡、多径传播以及定位误差等实际环境因素，与真实无线电传播环境仍存在一定差异。
- 2、滚动路径规划主要依据当前信息进行局部决策，在目标数量较多或分布复杂时，所得路线未必达到全局最优。

10.3 模型推广：干扰源允许出现区域尺度变化

题设中，干扰源只能出现在以原点为中心、半径 1800 m 的圆形区域内。为考察模型对不同任务空间尺度的适应能力，本文将**干扰源允许出现区域的半径**记为

$$R,$$

即干扰源真实位置 g 满足

$$g \in D_R = \{x \in \mathbb{R}^2 : \|x\| \leq R\}.$$

这里的 R 仅描述干扰源可能出现的空间范围，与干扰源的有效接收半径 $r \in [1000, 1500]$ m 以及七站结构中的外围扫描站半径 a 均为不同参数。

在保持有效接收半径、移动速度、检测时间、切频时间和清除规则不变的条件下，进一步令 $R = 1200, 1800, 2400, 3000$ m, 并重新生成与区域尺度相匹配的覆盖站布局、初始位置可行域及几何参数。

参数化版本仍沿用第三问框架。当给定区域半径 R 下，中心点和六个外围扫描站仍能满足最小接收半径 1000 m 条件下的全区域覆盖时，采用七站对称布局；当区域继续扩大、七站结构不再能够保证覆盖整个 D_R 时，程序根据新的区域半径自动生成满足覆盖距离条件的方格扫描站布局。

为验证上述参数化扩展，选取四个不同区域尺度的合成场景进行测试。各场景均包含 10 个干扰源，其空间位置由基准场景按照区域半径同比例缩放得到。结果如表 12 所示。

表 12 不同干扰源允许出现区域尺度下的参数化推广结果

允许区域半径 R/m	覆盖站数	清除结果	平均时间/(s/目标)
1200	7	10/10	193.89
1800	7	10/10	307.89
2400	21	10/10	474.14
3000	29	10/10	641.42

可以看到，当允许干扰源出现区域的半径由 1200 m 扩大至 1800 m 时，七站结构仍能够满足全区域覆盖要求；当区域半径进一步扩大到 2400 m 和 3000 m 时，原七站布局已不足以保证最小接收半径条件下的完整搜索，因此自动切换为方格覆盖布局，覆盖站数分别增加至 21 和 29 个。

四个代表场景均完成全部 10 个干扰源的清除，说明本文“覆盖搜索-集合定位-滚动规划-保证清除”的框架并不依赖于题设固定的 1800 m 区域半径。随着允许区域半径 R 增大，目标之间的平均空间距离、完整覆盖所需扫描站数量以及未知区域搜索成本均随之增加，因此平均定位清除时间由 193.89 s/目标逐步增加至 641.42 s/目标。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试、公式查询等，详细使用情况见支撑材料。

参考文献

- [1] Daniel Scholz. *Deterministic Global Optimization: Geometric Branch-and-Bound Methods and Their Applications*. Springer, 2012.
- [2] Pratap Tokekar and Volkan Isler. Sensor placement and selection for bearing sensors with bounded uncertainty. In *2013 IEEE International Conference on Robotics and Automation*, pages 2515–2520, 2013.
- [3] Godfried T. Toussaint. Solving geometric problems with the rotating calipers. In *Proceedings of IEEE MELECON '83*, Athens, Greece, 1983.
- [4] Heinrich W. E. Jung. Über den kleinsten kreis, der eine ebene figur einschließt. *Journal für die reine und angewandte Mathematik*, 137:310–313, 1910.
- [5] Michael Ian Shamos and Dan Hoey. Geometric intersection problems. In *Proceedings of the 17th Annual Symposium on Foundations of Computer Science*, pages 208–215, 1976.
- [6] Mark de Berg, Marc van Kreveld, Mark Overmars, and Otfried Schwarzkopf. *Computational Geometry: Algorithms and Applications*. Springer-Verlag, Berlin, 2 edition, 2000.
- [7] Kutluyıl Doğançay and Hatem Hmam. Optimal angular sensor separation for aoa localization. *Signal Processing*, 88(5):1248–1260, 2008.
- [8] Mauro Di Marco, Andrea Garulli, Simon Lacroix, and Antonio Vicino. Set membership localization and mapping for autonomous navigation. *International Journal of Robust and Nonlinear Control*, 11(7):709–734, 2001.
- [9] Aharon Ben-Tal, Laurent El Ghaoui, and Arkadi Nemirovski. *Robust Optimization*. Princeton University Press, 2009.
- [10] Ramon E. Moore, R. Baker Kearfott, and Michael J. Cloud. *Introduction to Interval Analysis*. Society for Industrial and Applied Mathematics, 2009.
- [11] Victor Pillac, Michel Gendreau, Christelle Guéret, and Andrés L. Medaglia. A review of dynamic vehicle routing problems. *European Journal of Operational Research*, 225(1):1–11, 2013.
- [12] Michael Held and Richard M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.
- [13] Peng Si Ow and Thomas E. Morton. Filtered beam search in scheduling. *International Journal of Production Research*, 26(1):35–62, 1988.
- [14] Stephen Boyd and Lieven Vandenbergh. *Convex Optimization*. Cambridge University Press, Cambridge, 2004.
- [15] Paolo Toth and Daniele Vigo, editors. *Vehicle Routing: Problems, Methods, and Applications*. Society for Industrial and Applied Mathematics, 2 edition, 2014.
- [16] Clair E. Miller, Albert W. Tucker, and Richard A. Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM*, 7(4):326–329, 1960.
- [17] Giordano Fusco and Himanshu Gupta. Selection and orientation of directional sensors for coverage maximization. In *2009 6th Annual IEEE Communications Society Conference on Sensor, Mesh and Ad Hoc Communications and Networks (SECON)*, pages 1–9. IEEE, 2009.

附录

附录 1: 支撑材料目录

表 13 支撑材料文件目录

编号	文件或目录	主要内容
S1	AI 工具使用详情.pdf	AI 工具使用情况及相关说明.
S2	q2/	问题二的完整支撑材料
S3	q2/一般案例/	问题二不同几何情形的数值实验.
S4	q2/已验证/	问题二全局优化与几何分支定界的验证证书
S5	q2/程序/	问题二候选区域构造、全局优化、几何计算及结果验证所使用的程序.
S6	q2/验证记录/	问题二各类算例的独立复核日志、环境与源码快照及证据一致性检查结果.
S7	q3/	问题三的完整支撑材料.
S8	q3/Q3_code/	问题三完整实现代码及运行、验证文件.
S9	q4/	问题四的完整支撑材料.
S10	q4/Q4_code/	问题四完整实现代码及运行、验证文件.
S11	数据统计数值验证/01_ 论文数据来源/	论文中主要实验数值的原始来源.
S12	数据统计数值验证/02_ 统计结果/	问题三 140 场实验与问题四 100 场随机实验的统计结果
S13	数据统计数值验证/03_ 数值验证/	论文关键结论的数值复核材料
S14	数据统计数值验证/README.md	数据来源、统计结果和数值验证材料的总体说明.

1.1 半平面交算法

Shamos 和 Hoey 对平面几何求交问题的系统研究, 给出了求 n 个半平面公共交的 $O(n \log n)$ 算法^[5]. 本文采用按边界方向角排序的半平面交方法: 首先将所有半平面的边界直线按方向角排序, 对于方向相同的平行约束仅保留限制更严格者; 随后利用双端队列维护能够构成当前可行区域边界的有效约束. 每加入一个新的半平面, 若队首或队尾相邻边界的交点落在该半平面之外, 则删除相应的无效边界.

全部约束处理完成后, 双端队列中保留下来的边界即为最终定位区域的有效边界, 相邻两条有效边界的交点依次构成凸多边形顶点 $V_0, V_1, \dots, V_{m-1}$. 由于半平面排序需要 $O(n \log n)$ 的时间, 而每条边界在双端队列中至多进入和删除一次, 因此算法总体时间复杂度为

$$O(n \log n).$$

因此, 相比适用于小规模问题的直接枚举法, 半平面交算法能够在保持求解准确性的同时, 对检测点数量增加的情况保持更稳定的计算性能.

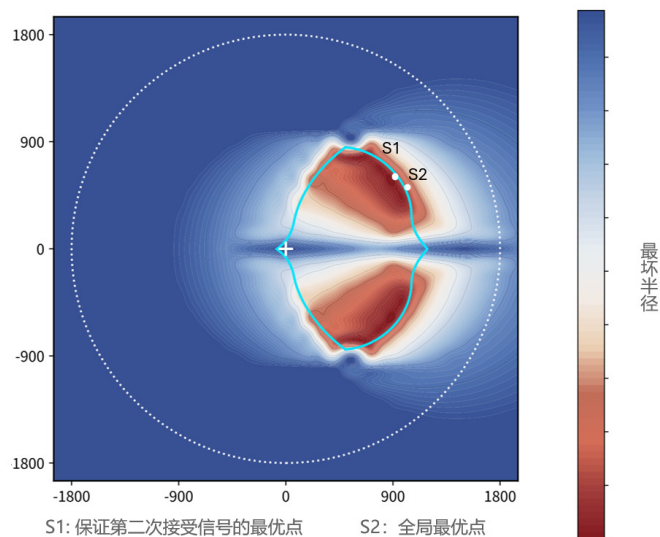


图 19 基准算例的全局与保证接收域推荐第二检测点

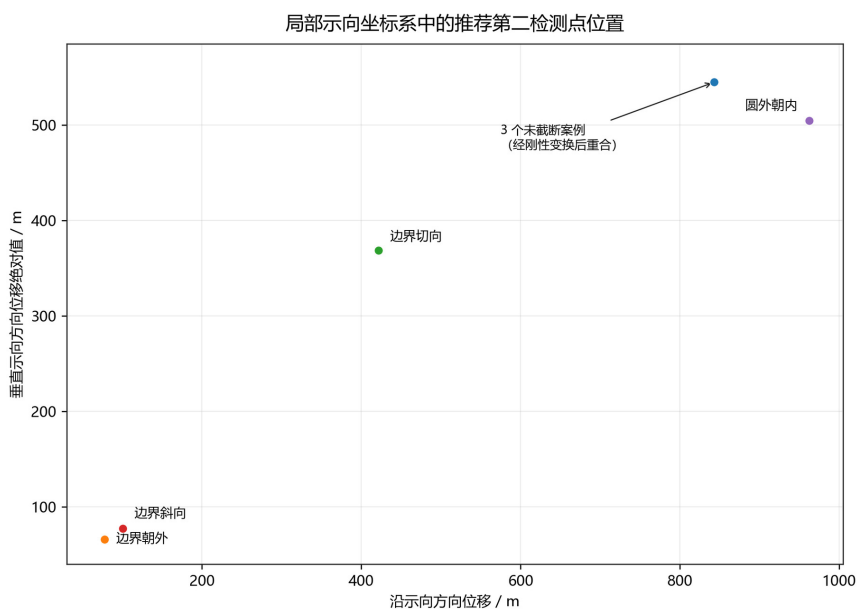


图 20 不同代表性算例中推荐第二检测点在首次示向局部坐标系中的位置

附录 2：问题三补充推导与算法细节

2.1 有限后备的时间与动作上界

首次获得阳性示向后，目标被限制在局部窄扇区

$$0 \leq x \leq 1500, \quad |y| < 27 \text{ m.}$$

正文采用 20 m 纵向间隔与五条横向扫描线

$$y \in \{-40, -20, 0, 20, 40\} \text{ m,}$$

共形成 380 个蛇形候选清除点。考虑 5 m 坐标量化后，候选点到真实源的最坏距离不超过

$$\frac{20}{\sqrt{2}} + \frac{5}{\sqrt{2}} = 17.678 \text{ m} < 20 \text{ m,}$$

从而保证有限候选点中至少存在一个可成功清除的位置。

设当前尚未排除的未知频道数为 u ，尚未完成的覆盖站集合为 Q_{pending} ，已确认的实际源数为 k ，源总数上限为 $n_{\text{max}} = 16$ 。对剩余任务采用保守计数，可写出后备时间与动作数上界

$$U_{\text{fb}} = T_{\text{pending}} + 6u|Q_{\text{pending}}| + 5000(n_{\text{max}} - k), \quad (49)$$

$$A_{\text{fb}} = u|Q_{\text{pending}}| + 380(n_{\text{max}} - k). \quad (50)$$

其中 T_{pending} 为完成剩余覆盖站所需的移动时间上界。程序在每次普通决策前利用上述量检查剩余预算；若继续执行普通动作会破坏后备可完成性，则立即锁定后备流程。

在初始状态下，对应的保守上界为

$$U_{\text{fb}} = 82219.517236 \text{ s}, \quad A_{\text{fb}} = 6220.$$

2.2 联合任务-服务模式动态规划

设任务组 j 的候选服务模式集合为 $\mathcal{M}_j$ ，模式可对应补充测量、试探清除或可靠清除。令 U 为尚未处理的任务组集合， i 为上一服务模式， q 为当前接收频道，则正文中的联合规划递推为

$$F(U, i, q) = \min_{\substack{j \in U \\ m \in \mathcal{M}_j}} [d_{im} + \tau_m(q) + F(U \setminus \{j\}, m, q'_m(q))],$$

其中

$$F(\emptyset, i, q) = 0.$$

这里 d_{im} 为服务模式之间的移动时间， $\tau_m(q)$ 同时计入模式内部操作时间和切频成本， $q'_m(q)$ 为执行模式后的频道状态。因此一次递推同时确定“下一任务”和“该任务采用的服务方式”。

任务组数不超过 7 时采用完整子集动态规划；任务组较多时采用宽度 64 的束搜索。束搜索对部分路径综合已发生代价、剩余服务成本以及最小生成树松弛下界进行排序，并始终保留完整可行参考方案。每次只执行当前规划的第一个任务，获得新反馈后重新构造任务集合并求解。

2.3 连续可靠清除块的上下界证书

固定目标访问顺序后，设清除块入口和出口分别为 A, B ，第 i 个目标的可靠清除区域为 C_i ，连续服务点优化问题为

$$f^* = \min_{y_i \in C_i} \left(\|A - y_1\| + \sum_{i=1}^{m-1} \|y_i - y_{i+1}\| + \|y_m - B\| \right).$$

由于 C_i 为圆盘交形成的凸集，且目标函数为欧氏范数之和，该问题为凸优化问题。

为处理零距离处的不可微性，采用

$$f_\varepsilon(y) = \sum_{e=1}^E \sqrt{\|d_e\|^2 + \varepsilon^2}, \quad \varepsilon = 10^{-5} \text{ m},$$

其中 E 为清除块路径边数，并有

$$f_\varepsilon - E\varepsilon \leq f \leq f_\varepsilon.$$

在当前可行点 y 处，记

$$g_i = \nabla_{y_i} f_\varepsilon(y), \quad \ell_i \leq \min_{z \in C_i} g_i^\top z.$$

由凸函数的一阶支撑不等式，可构造

$$L = f_\varepsilon(y) - E\varepsilon + \sum_i (\ell_i - g_i^\top y_i) \leq f^* \leq U = f(y^{\text{feas}}).$$

因此 $U - L$ 直接给出固定顺序连续子问题的最优性间隙。

数值求解最多迭代 160 次，目标间隙设为 0.002 m. 140 场演练共记录 285 个连续清除块，其中 209 个达到该目标，占 73.33%；其余 76 个仍保留相应的可行上界和支撑下界. 全部结果的间隙中位数为

$$7.82 \times 10^{-4} \text{ m},$$

P90 为

$$1.03 \times 10^{-2} \text{ m},$$

平滑近似误差上界最大为

$$3 \times 10^{-5} \text{ m}.$$

优化得到的连续坐标在执行前重新量化到题目要求的坐标网格，并重新检验可靠清除条件；随后按整数微秒重算清除块成本，仅当新方案不增加原清除块成本时才采用.

2.4 140 场分层统计明细

140 场演练按目标数 $N = 10, \dots, 16$ 分层，每档 20 场，共清除 1820 个干扰源. 各层结果见表 14.

表 14 问题三 140 场演练的分层统计

N	场数	均值/(s/目标)	标准差/(s/目标)	P90/(s/目标)
10	20	274.23	22.14	295.48
11	20	265.03	22.66	293.64
12	20	252.94	21.58	280.89
13	20	233.25	20.94	258.58
14	20	218.93	15.55	240.51
15	20	209.88	14.17	230.20
16	20	196.36	19.13	217.33

总体平均定位清除时间为 235.80 s/目标，标准差 33.34 s/目标，中位数 231.22 s/目标，P90 为 282.48 s/目标. 时间组成中，移动、检测、切频、成功清除和失败清除分别约占 78.02%、16.21%、3.07%、2.16% 和 0.55%.

2.5 离线时间下界的构造

任务结束后利用真实源位置 G_i 构造理想化清除下界. 原点至第 i 个 20 m 清除圆盘的边距离取为

$$\underline{d}_{0i} = \max\{0, \|G_i\| - 20\},$$

两个清除圆盘之间取

$$\underline{d}_{ij} = \max\{0, \|G_i - G_j\| - 40\}.$$

在该有限图上利用 Held-Karp 算法求原点出发、终点自由的最短开放路径 L_{HK} ，并加入每个目标至少一次成功清除所需的 5 s，得到

$$T_{LB} = \frac{L_{HK}}{5} + 5N.$$

140 场的平均下界为 132.67 s/目标，实际平均为 235.80 s/目标. 该下界用于给出整体时间尺度参考.

附录 3：问题四补充推导与实验结果

本附录补充正文中因篇幅限制省略的位置-方向覆盖证明、联合规划完整约束、有限后备推导及部分实验结果.

3.1 位置-方向覆盖的详细证明

正文给出了固定源位置 g 的全朝向可见性判据. 记

$$S_g = \{s \in S : \|s - g\| \leq 1000\}.$$

若 $g \in \text{conv}(S_g)$, 则存在

$$g = \sum_i \lambda_i s_i, \quad \lambda_i \geq 0, \quad \sum_i \lambda_i = 1.$$

于是对任意发射方向 $u \in \mathbb{S}^1$,

$$\sum_i \lambda_i (s_i - g)^\top u = 0,$$

故至少存在一个 $s_i \in S_g$ 满足

$$(s_i - g)^\top u \geq 0.$$

因此该扫描点同时满足距离条件和发射半平面条件.

反之, 若

$$g \notin \text{conv}(S_g),$$

由严格分离定理, 存在方向 u 使

$$(s - g)^\top u < 0, \quad \forall s \in S_g.$$

于是该发射方向下所有最小接收半径内的扫描点均位于发射背面, 无法保证检测到该源. 故有

$$\forall u \in \mathbb{S}^1, \exists s \in S_g : (s - g)^\top u \geq 0 \iff g \in \text{conv}(S_g).$$

进一步, 对三角形单元 T 选择扫描点集合 W_T . 若

$$T \subseteq \text{conv}(W_T), \quad \max_{s \in W_T} \max_{v \in V(T)} \|s - v\| < 1000,$$

则对任意 $g \in T$, 由于距离函数关于 g 为凸函数, 其在三角形上的最大值出现在顶点, 故

$$\|s - g\| < 1000, \quad \forall s \in W_T.$$

同时

$$g \in \text{conv}(W_T) \subseteq \text{conv}(S_g),$$

因而整个单元满足全朝向可见性条件.

基础 21 点由原点、半径 995 m 的 8 个等分内圈点和 12 个外圈点组成. 外圈正十二边形的设计内切圆半径为 1801 m, 对应顶点半径约 1864.53 m. 本文通过自适应三角剖分, 对各叶单元逐一检验上述凸包包含条件和距离条件, 从而验证 21 点布局在连续目标区域上的位置-方向覆盖充分性. 21 点是本文采用的一个经验验证可行布局, 并不代表最少扫描点数.

原点扫描后允许整体旋转非原点站点. 若 $R(\alpha)$ 为旋转矩阵, 则

$$\|R(\alpha)q - g\| = \|q - R(\alpha)^T g\|,$$

且

$$(R(\alpha)q - g)^\top u = (q - R(\alpha)^T g)^\top R(\alpha)^T u.$$

由于目标区域为圆域, 理想旋转不改变上述位置-方向覆盖性质; 实际部署时, 旋转坐标量化后重新检验目标圆包含、凸包包含和距离条件.

3.2 覆盖与服务联合规划的完整约束

3.2.1 剩余覆盖路径优化

设 I 为当前尚可选择的扫描站集合, $y_i \in \{0, 1\}$ 表示是否访问站点 i , $x_{ij} \in \{0, 1\}$ 表示是否使用无向路径边 $\{i, j\}$, $e_i \in \{0, 1\}$ 表示站点 i 是否为开放路径终点, $z_k \in \{0, 1\}$ 表示是否启用第 k 个凸包联合覆盖条件. 目标函数为

$$\min \left[\frac{1}{5} \sum_{\{i,j\} \in E} d_{ij} x_{ij} + \sum_{i \in I} \tau_i y_i \right].$$

若第 k 个联合覆盖条件所需的扫描站集合为 W_k , 则

$$z_k \leq y_i, \quad \forall i \in W_k.$$

对每个尚未满足的覆盖要求 ℓ , 设 D_ℓ 为能够直接满足该要求的扫描站集合, K_ℓ 为能够联合满足该要求的覆盖条件集合, 则

$$\sum_{i \in D_\ell} y_i + \sum_{k \in K_\ell} z_k \geq 1.$$

已经由历史观测完成的覆盖要求在建立当前模型前删除.

把当前位置记为源点 0 , 则开放路径满足

$$\begin{aligned} \sum_{j: \{i,j\} \in E} x_{ij} &= 2y_i - e_i, \quad i \in I, \\ \sum_{j: \{0,j\} \in E} x_{0j} &= 1, \quad \sum_{i \in I} e_i = 1. \end{aligned}$$

对不含源点的候选连通分量 C , 采用割约束

$$\sum_{\{i,j\} \in \delta(C)} x_{ij} \geq y_k, \quad k \in C,$$

逐步排除与当前位置断开的子环. 整数路线得到后, 再次检验连续位置-方向覆盖条件.

3.2.2 服务方式-访问顺序优化

对每个冻结目标组 g 设候选服务模式集合为 $\mathcal{M}_g$, 模式选择变量为 y_m , 模式图上的有向边变量为 a_{mn} . 每个目标组恰选择一个当前服务模式:

$$\sum_{m \in \mathcal{M}_g} y_m = 1.$$

被选择模式满足流守恒, 并设置唯一源边和唯一终点边; 组顺序变量采用 MTZ 型约束消除与源点断开的子环.

模式 m 的实际服务时间依赖进入该模式前的接收频道 q . 第一项使用当前真实频道成本; 对于后续模式, MILP 采用

$$\tau_m = \min_q \tau_m(q)$$

作为未来频道成本松弛. MILP 返回候选顺序后, 从真实当前频道开始逐项恢复频道状态并重新计算

$$U_t = \sum_k [d_{m_{k-1}, m_k} + \tau_{m_k}(q_{k-1})].$$

该值对应一条可执行候选路线. 结合经过一致性检查的 MILP 有效下界, 得到

$$L_t \leq J_t^{*, \text{cat}} \leq U_t,$$

其中 $J_t^{*, \text{cat}}$ 仅表示当前冻结有限候选目录上的最优预测成本.

3.3 有限后备与条件完备性

后备格点间距为

$$h = 25 \text{ m.}$$

任意平面点到最近格点的最大距离为

$$\frac{h}{\sqrt{2}} = 17.68 \text{ m} < 20 \text{ m.}$$

首次正反馈后形成的位置外包为有界集合，因此与该外包相交或距离不超过 $25/\sqrt{2}$ 的 25 m 格点数量有限。失败格点永久去重，不再重复尝试；只要位置外包保持保守，真实源附近至少存在一个满足 20 m 清除条件的候选格点。

设以下条件成立：观测符合题设物理规则；位置外包始终包含真实位置；剩余覆盖任务和已发现目标均获得公平服务；后备格点不重复；运行时间和动作资源足以执行这些有限任务。

在这些条件下，21 点布局的连续位置-方向覆盖性质保证每个真实源至少对应一个可见扫描位置，因此公平执行剩余覆盖任务后所有真实源都会被发现。已发现目标若形成可靠清除区域即可直接完成；否则在达到局部阈值后进入有限 25 m 格点后备，并在有限次不重复尝试中完成清除。由于真实源总数最多 16 个，整个任务由有限个覆盖任务和有限个目标服务过程组成。

3.4 补充实验结果

100 场本地随机仿真实验中，服务方式-路径联合 MILP 共参与 3934 次滚动决策，其中 122 次改变基础方案的首动作，占 3.10%；42 场没有发生首动作改写；3934 次求解中有 407 次在当前有限候选目录上达到数值零间隙判据。上述计数用于描述联合层的实际参与程度，并不构成同场景消融结果。

目标中心开放最短路参考时间平均为

$$1869.99 \text{ s,}$$

考虑 20 m 清除圆距离松弛后的物理时间下界平均为

$$1771.45 \text{ s,}$$

实际虚拟总时间均值为

$$5615.69 \text{ s.}$$

三者的差异综合反映了覆盖搜索、主动定位、频道切换、在线路径选择以及距离松弛产生的附加代价。

三次正式测试的具体成绩与代表性路线已在正文给出。图图 24 补充展示三场测试的时间与路程尺度。

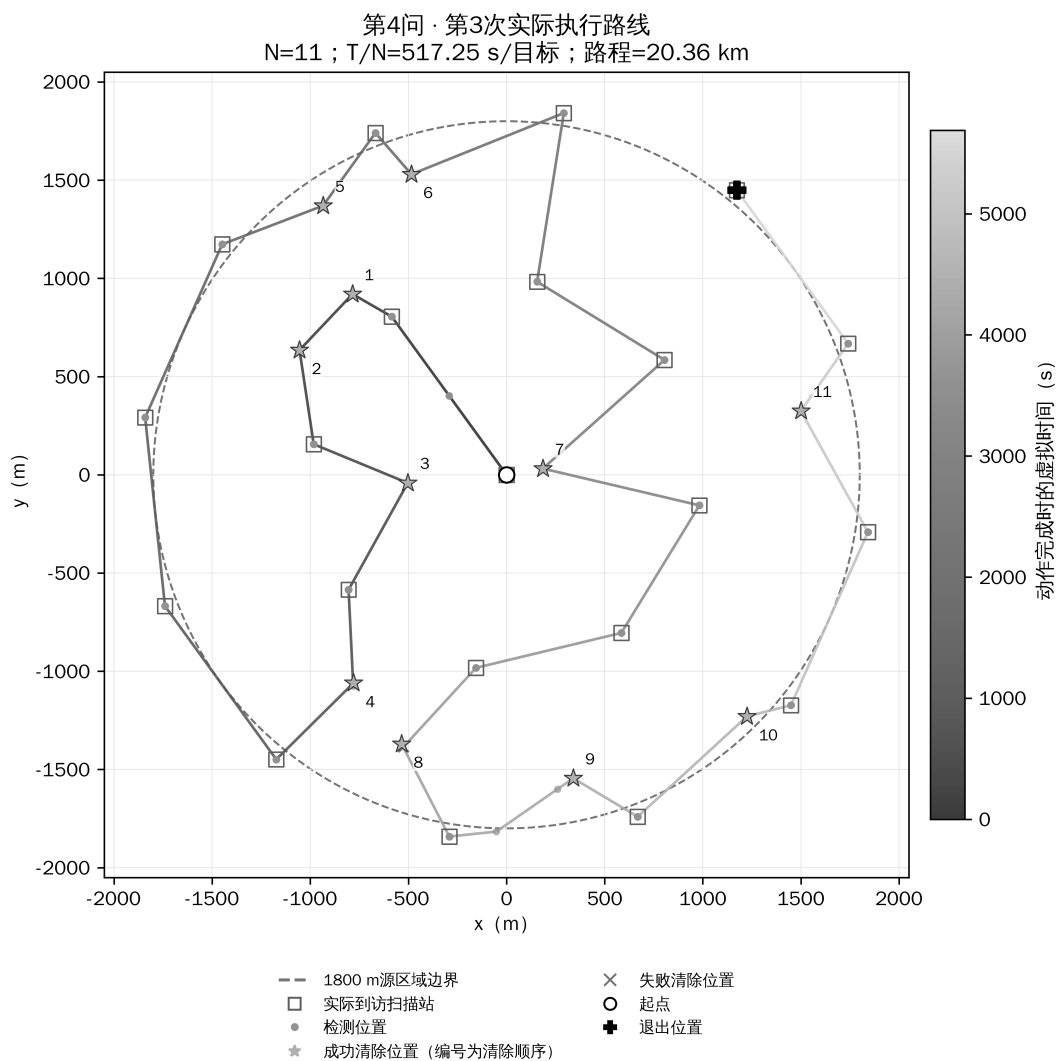


图 21 问题四第 3 次正式测试的实际运行路线

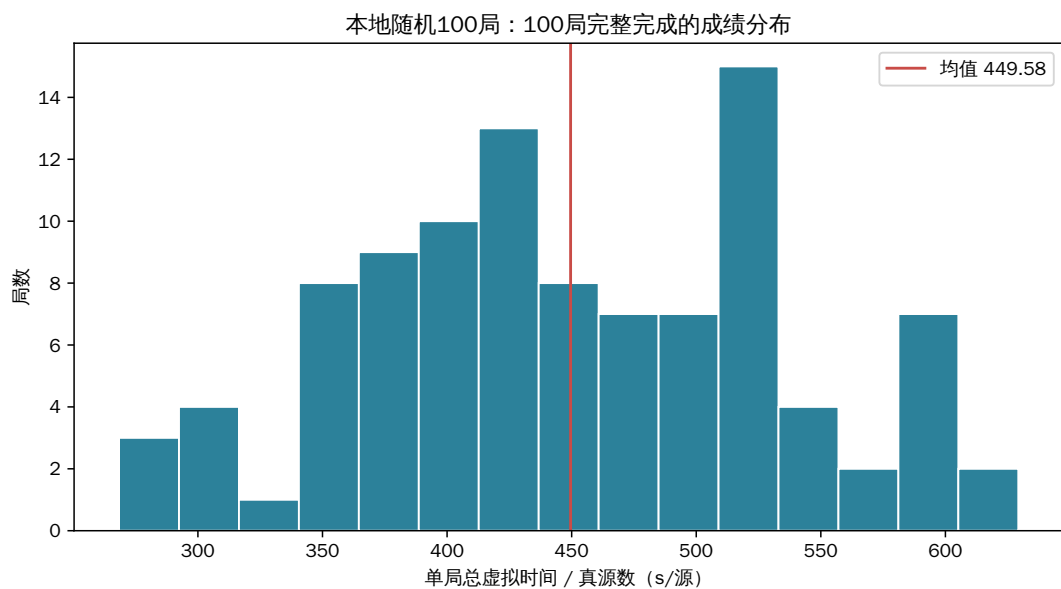


图 22 问题四 100 场本地随机仿真实验的单位目标耗时分布

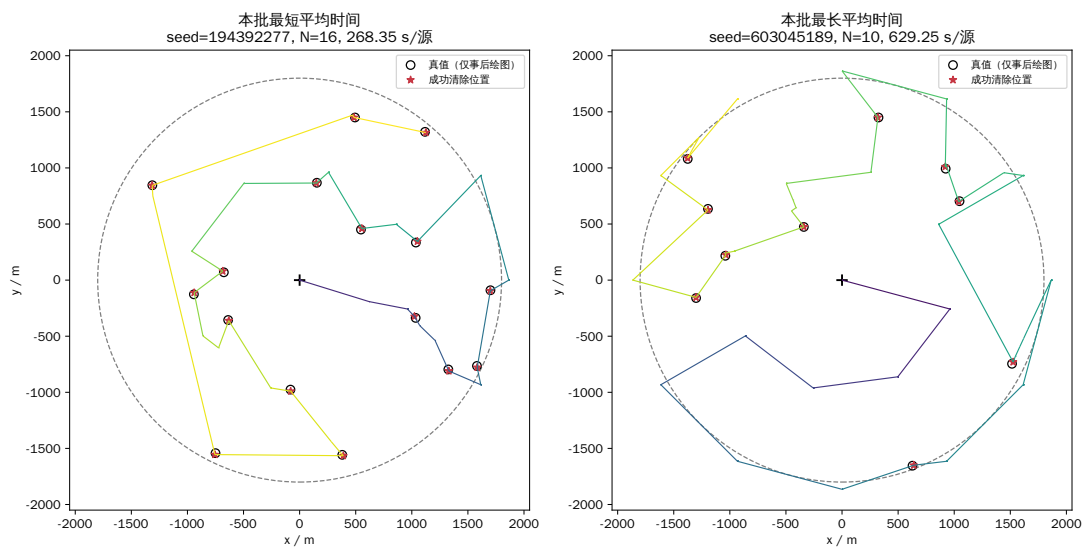


图 23 问题四 100 场本地随机仿真实验的典型快慢案例

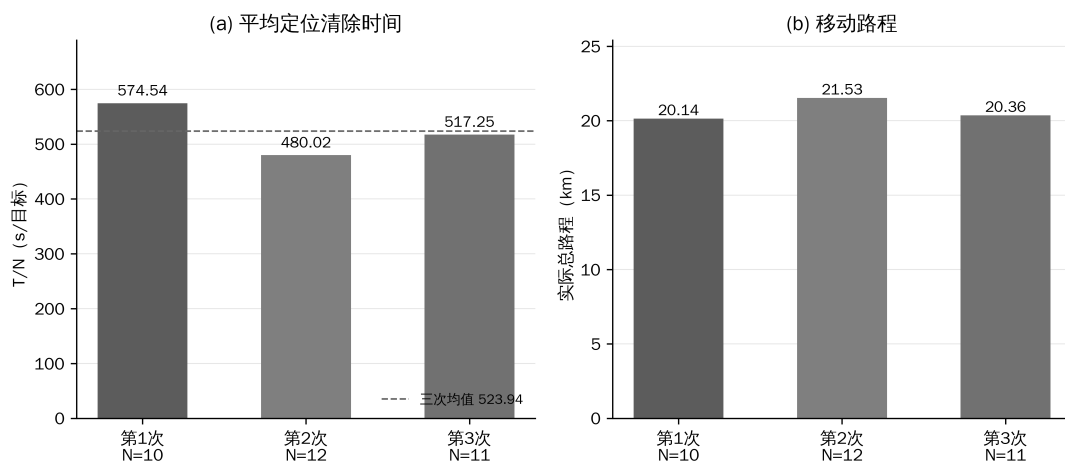


图 24 问题四三次正式测试的单位目标耗时与执行路程

Listing 1: Q2 源码

```

def candidate(region):
    """边界搜索生成候选点。"""
    d = math.radians(1)

    def pos(beta):
        b = math.radians(beta)
        return (
            5 * math.cos(d) + 999.99999 * math.cos(b),
            -5 * math.sin(d) + 999.99999 * math.sin(b),
        )

    history = []

    def objective(beta):
        p = pos(beta)
        if not feasible_exact(point(p)):
            return 10000.
        value = bounds(region, p, tolerance=.002)['upper_m']
        history.append((value, p, beta))
        return value

    minimize_scalar(
        objective,
        bounds=(25, 45),
        method='bounded',
        options={'xatol': 1e-5, 'maxiter': 45},
    )

    value, p, beta = min(history)
    assert feasible_exact(point(p))
    return value, p, history

```

```
[language=Markdown,caption={Q3源码}, label={lst:q3mdexample}]
```

1. 集合定位与保证清除

正示向采用半平面裁剪；负反馈单独记录；保证清除点需覆盖整个保守多边形。

```
### `bearing_clip`
```

来源：`Q3\_submission/src/solver/geometry.py`，第48—54行。

```

```python
def bearing_clip(poly, position, bearing_deg, epsilon_deg=EPS_DEG):
 lower, upper = [math.radians(bearing_deg + sign * epsilon_deg) for sign in (-1, 1)]
 for normal in ((math.sin(lower), -math.cos(lower)),
 (-math.sin(upper), math.cos(upper))):
 offset = normal[0] * position[0] + normal[1] * position[1]
 poly = clip(poly, normal, offset + 1e-7)
 return poly
```

```

```
### `clear_position`
```

来源：`Q3\_submission/src/solver/geometry.py`，第134—161行。

```

```python
def clear_position(poly, current, radius=CLEAR_SAFE_RADIUS_M):

```

```

"""Closest point in intersection of vertex-centered clearance disks."""
center, r = enclosing_circle(poly)
if r > radius:
 return None
def feasible(p):
 return all(distance(p, v) <= radius + 1e-7 for v in poly)
if feasible(current):
 return current
candidates = [center]
for v in poly:
 d = distance(v, current)
 if d:
 p = (v[0]+radius*(current[0]-v[0])/d, v[1]+radius*(current[1]-v[1])/d)
 if feasible(p):
 candidates.append(p)
 for i, a in enumerate(poly):
 for b in poly[i:]:
 d = distance(a, b)
 if d < 1e-9 or d > 2*radius:
 continue
 mid = ((a[0]+b[0])/2, (a[1]+b[1])/2)
 h = math.sqrt(max(0, radius*radius-d*d/4))
 for sign in (-1, 1):
 p = (mid[0]+sign*h*(b[1]-a[1])/d, mid[1]-sign*h*(b[0]-a[0])/d)
 if feasible(p):
 candidates.append(p)
return min(candidates, key=lambda p: distance(p, current))
...

```

### `Belief.observe`

来源: `Q3\_submission/src/solver/belief.py`, 第30—62行。

```

...python
def observe(self, position, response, anchor_index=None):
 kind = response["measure_result"]
 self.physical_history.append(('measure', tuple(position), kind, response.get('
 svd_deg')))
 self.observations.append((tuple(position), kind, response.get("svd_deg")))
 if anchor_index is not None:
 self.covered.add(anchor_index)
 if kind == "no_signal":
 self.negatives.append(tuple(position))
 if self.state == "found":
 self.misses += 1
 self.local_measurements += 1
 self.sensing_cache = None
 return kind
 if kind not in ("direction", "near"):
 raise ValueError(f"Unknown measure_result: {kind}")
 self.state = "found"
 old_radius = self.radius
 if kind == "near":
 poly = clip_disk_outer(self.polygon, position, 5)
 else:
 poly = bearing_clip(self.polygon, position, response["svd_deg"])
 poly = clip_disk_outer(poly, position, 1500)
 if not poly:
 raise ValueError(f"Channel {self.channel}: inconsistent bearing region; refusing
 false clearance certificate")

```

```

self.polygon = poly
self.center, self.radius = enclosing_circle(poly)
self.stagnant = self.stagnant + 1 if self.radius > .85*old_radius else 0
self.misses = 0
self.local_measurements += 1
self.revision += 1
self.sensing_cache = None
self.fallback_cache = None
return kind
```

```

2. 扫描覆盖排除

三角单元三个顶点都落在同一接收圆内才标记覆盖。

```

### `Coverage.mask`

```

来源: `Q3\_submission/src/discrete\_model/coverage.py`, 第36—38行。

```

```python
def mask(self,p):
inside=np.all(np.sum((self.vertices-np.array(p))**2,axis=2)<999.9999**2,axis=1)
return bits(inside)
```

```

```

### `Coverage.observe_scan`

```

来源: `Q3\_submission/src/discrete\_model/coverage.py`, 第45—47行。

```

```python
def observe_scan(self,p):
self.remaining &= ~self.mask(p)
self.scanned.append(tuple(p))
```

```

3. 搜索、测量与清除的联合滚动决策

这段是第三问上层核心: 生成每个目标的多种候选执行方案, 同时加入待扫描站, 再联合决定下一动作。不是只选最近目标。

```

### `CoupledController.next_job`

```

来源: `Q3\_submission/src/nested\_joint/no\_score/controller.py`, 第37—95行。

```

```python
def next_job(self):
before=self.joint_stats['plans'];original=super().next_job()
unknown=[c for c,b in self.beliefs.items() if b.state=='unknown']
if self.joint_stats['plans']==before or self.locked is not None or (unknown and
self.since_scan>=18):return original
found=[b for b in self.beliefs.values() if b.state=='found']
if any(len(b.failed_clears)>=8 or b.local_measurements>=10 for b in found):return
original
plan=self.joint.latest;scans=sorted(plan['scans'])
if len(found)+len(scans)<2:return original
try:clouds={b.channel:self.posterior.cloud(b) for b in found}
except PosteriorUnavailable:return original
centres={c:snap(cloud.mean) for c,cloud in clouds.items()}
groups=[];representatives=[];first=None
for b in found:

```

```

c=b.channel;cloud=clouds[c];g=len(groups);jobs=[];current=self.client.position
def add(kind,p,guaranteed=False):
 p=tuple(p)
 if math.hypot(*p)>(3300 if kind=='measure' else 1820):return
 if kind=='measure' and any(distance(p,q)<.1 for q,_,_ in b.observations):return
 if not any(j.kind==kind and j.point==p for j in jobs):jobs.append(Job(kind,p,c,
 guaranteed))
 if original.channel==c and original.kind in ('measure','clear'):jobs.append(
 original)
 add('clear',centres[c])
 good=self.reliable_options(b)
 if good:
 ends=[current,*sorted(scans,key=lambda p:distance(p,centres[c]))[:1]]
 for end in ends:add('clear',min(good,key=lambda p:distance(p,end)),True)
 proposals=[centres[c],snap(current),*map(snap,cautious_points(cloud))]
 # Shared arrival points permit scan -> localization with zero extra
 # travel. They are candidates, never assumed to produce a bearing.
 proposals+=sorted(scans,key=lambda p:distance(p,centres[c]))[:2]
 proposals+=sorted((p for other,p in centres.items() if other!=c),key=lambda p:
 distance(p,centres[c]))[:2]
 if getattr(self, "clearability_enabled", False):
 from clearability import proposals as gap_proposals
 proposals += gap_proposals(self,b,proposals)
 for p in proposals:add('measure',p)
 modes=[self.cached_mode(g,job,cloud) for job in jobs]
 if not modes:return original
 if original.channel==c:first=(g,0)
 groups.append(modes);representatives.append(centres[c])
 for p in scans:
 g=len(groups);job=Job('scan',p,anchor=self.anchors.index(p) if p in self.anchors
 else None,scan_channels=tuple(unknown))
 if original.kind=='scan' and tuple(original.point)==tuple(p):job=original;first=g,0
 groups.append([Mode(g,job,0.,np.array([p]),np.array([1.]));representatives.
 append(p)
 if first is None:return original
 problem=self.route_problem_class(groups,self.client.position,self.client.channel)
 refindex=problem.indices[first[0]][first[1]]
 remaining=[g for g in range(len(groups)) if g!=first[0]]
 remaining.sort(key=lambda g:plan['path'].index(representatives[g]) if
 representatives[g] in plan['path'] else len(plan['path']))
 indices,proof=problem.solve(refindex,[first[0],*remaining])
 selected=problem.modes[indices[0]].job
 changed=(selected.kind,selected.channel,selected.point)!=(original.kind,original.
 channel,original.point)
 self.coupled_stats['decisions']+=1;self.coupled_stats['changed_actions']+=changed
 self.coupled_stats['exact_catalog_solves' if proof['exact_catalog'] else '
 beam_solves']+=1
 self.coupled_stats['expanded']+=proof['expanded'];self.coupled_stats['
 modelled_savings_s']+=proof['reference_s']-proof['selected_s']
 self.joint_stats['scan_jobs']+=int(selected.kind=='scan')-int(original.kind=='scan'
 ')
 self.client.record('coupled_route_decision',original=original.__dict__,selected=
 selected.__dict__,
 plan=[dict(job=problem.modes[i].job.__dict__,internal_s=problem.modes[i].
 internal_s,
 exits=problem.modes[i].exits.tolist(),weights=problem.modes[i].weights.tolist())
 for i in indices],
 **proof,original_problem_optimality=False,episode_nonincrease_guaranteed=False)
 return selected

```

```

4. 含模式与接收频道的DP及束搜索

状态保留未完成任务集合、上一候选执行方案和接收频道；最多7个任务组完整DP，更多时束宽64。这里的界只属于有限冻结服务模型。

`RouteProblem.lower`

来源：`Q3\_submission/src/coupled\_route/routing.py`，第40—44行。

```
```python
def lower(self,mask,last):
 if not mask:return 0.
 groups=[g for g in range(self.n) if mask>>g&1]
 incoming=self.first if last<0 else self.move[last]
 return sum(self.group_service[g] for g in groups)+self.tree(mask)+min(float(
 incoming[i]) for g in groups for i in self.indices[g])
```
```

`RouteProblem.transitions`

来源：`Q3\_submission/src/coupled\_route/routing.py`，第46—51行。

```
```python
def transitions(self,mask,last,q):
 incoming=self.first if last<0 else self.move[last]
 for g in range(self.n):
 if mask>>g&1:
 for i in self.indices[g]:
 yield mask^(1<<g),i,int(self.receiver_out[i,q-1]),float(incoming[i]+self.service[i
 ,q-1])
```
```

`RouteProblem.solve`

来源：`Q3\_submission/src/coupled\_route/routing.py`，第65—97行。

```
```python
def solve(self,reference_first,reference_order,beam_width=64):
 original=self.fixed_order(reference_order,reference_first)
 best=original;full=(1<<self.n)-1;lower=self.lower(full,-1)
 if self.n<=7:
 @lru_cache(None)
 def dp(mask,last,q):
 if not mask:return 0.,(,)
 answer=(math.inf,())
 for left,i,q2,cost in self.transitions(mask,last,q):
 tail,path=dp(left,i,q2);self.expanded+=1
 if cost+tail<answer[0]:answer=cost+tail,(i,)+path
 return answer
 best=dp(full,-1,self.receiver);lower=best[0];exact=True
 else:
 # A fixed-width search is approximate even with admissible ranking.
 states={full,-1,self.receiver):(0.,())}
 for depth in range(self.n):
 nxt={}
 for (mask,last,q),(cost,path) in states.items():
 for left,i,q2,step in self.transitions(mask,last,q):
 self.expanded+=1;value=cost+step;key=left,i,q2
```
```

```

if value+self.lower(left,i)>=best[0]-1e-9:continue
if key not in nxt or value<nxt[key][0]:nxt[key]=value,path+(i,)
if not nxt:break
if depth==self.n-1:
trial=min(nxt.values(),key=lambda x:x[0])
if trial[0]<best[0]:best=trial
break
states=dict(sorted(nxt.items(),key=lambda item:item[1][0]+self.lower(item[0][0],
    item[0][1]))[:beam_width])
exact=False
assert best[0]<=original[0]+1e-6
assert lower<=best[0]+1e-6
return best[1],dict(reference_s=original[0],selected_s=best[0],lower_s=lower,gap_s
    =max(0,best[0]-lower),exact_catalog=exact,expanded=self.expanded,groups=self.n
    ,modes=len(self.modes),scope='finite frozen service modes; not complete online
    time bound')
...

```

5. 连续保证清除区域的上下界优化

固定顺序、入口及出口后优化服务点。输出可行长度、支撑下界和实际间隙；量化后重新核验，只有不增加原块成本才采用。

`fixed\_order`

来源：`Q3\_submission/src/region\_route/clearance.py`，第14—57行。

```

```python
def fixed_order(tasks,start,end=None,iterations=160,tolerance_m=.002):
regions=[clearance_region(tuple(map(tuple,t['polygon']))),RADIUS) for t in tasks]
points=[tuple(t['point']) for t in tasks]
if any(not r.feasible(p,1e-9) for r,p in zip(regions,points)):
raise ValueError('Continuous incumbent is not reliable')
if end is not None:points.append(tuple(end))
best=list(points);upper=path_length(best,start);lower=0.;epsilon=1e-5;used=0
for used in range(1,iterations+1):
value,gradient=smooth_value_gradient(points,start,epsilon)
bound=value-len(points)*epsilon
directions=[(0.,0.) for _ in points]
for i,region in enumerate(regions):
proposal,support=region.linear_minimum(gradient[i])
directions[i]=minus(proposal,points[i])
bound+=support-dot(gradient[i],points[i])
lower=max(lower,bound)
if lower>upper+1e-5:raise ValueError('Inconsistent convex bounds')
if upper-lower<=tolerance_m:break
step=line_search(points,directions,start,epsilon)
if step==0:break
points=[tuple(p[k]+step*d[k] for k in (0,1)) for p,d in zip(points,directions)]
for i,r in enumerate(regions):points[i]=r.repair(points[i])
cost=path_length(points,start)
if cost<upper:best=list(points);upper=cost
Quantize for the actual interface, then independently check all vertices.
A failed rounding check retains the corresponding original feasible point.
quantized=[]
for t,r,p in zip(tasks,regions,best):
q=tuple(round(x,6) for x in p)
if not reliable(t['polygon'],q):
p=tuple((1-1e-7)*x+1e-7*c for x,c in zip(p,r.center))
q=tuple(round(x,6) for x in p)

```

```

if not reliable(t['polygon'],q):q=tuple(t['point'])
quantized.append(q)
length=path_length(quantized+([tuple(end)] if end is not None else []),start)
info=dict(method='continuous_conditional_gradient_with_disk_support_dual',
scope='fixed order, reliable regions, fixed entry and endpoint',
lower_length_m=lower,feasible_quantized_length_m=length,
gap_m=max(0.,length-lower),gap_target_m=tolerance_m,
gap_target_met=length-lower<=tolerance_m,iterations=used,
smoothing_error_bound_m=len(points)*epsilon,
floating_point_guarded=True,interval_arithmetic_certified=False,
global_online_optimality=False)
return quantized,info
'''

```

### `optimize`

来源: `Q3\_submission/src/region\_route/clearance.py`, 第60—82行。

```

'''python
def optimize(tasks,start,end=None):
 finite,proof=finitize_optimize(tasks,start,end)
 if not finite:return finite,proof
 ordered=[dict(tasks[t['original_index']],point=t['point']) for t in finite]
 points,info=fixed_order(ordered,start,end)
 proposed=[dict(t,point=p) for t,p in zip(finite,points)]
 proposed_us=path_us(points,start,end)
 finite_us=proof['new_us']
 selected=proposed if proposed_us<finite_us else finite
 selected_us=min(proposed_us,finite_us)
 edges=len(tasks)+int(end is not None)
 # At most 0.5 microsecond rounding per edge, with an extra numerical guard.
 lb_us=math.floor(info['lower_length_m']/5*1e6-edges*.5-1)
 assert selected_us<=finite_us<=proof['old_us']
 assert all(reliable(tasks[t['original_index']]['polygon'],t['point']) for t in
 selected)
 proof.update(new_us=selected_us,saved_us=proof['old_us']-selected_us,
 finite_incumbent_us=finite_us,continuous_saved_us=finite_us-selected_us,
 continuous_refinement=info,selected_order_lower_us=lb_us,
 selected_order_gap_us=max(0,selected_us-lb_us),
 exact_order_in_candidates=False,
 finite_incumbent_exact_order_in_candidates=proof['exact_order_in_candidates'],
 scope='same certified clear block and endpoints; finite incumbent retained')
 return selected,proof
'''

```

## 6. 蛇形遍历清除与剩余资源检查

根据首次定向结果构造380个候选清除点，并按蛇形顺序访问。  
 每次执行常规任务前检查剩余时间和动作次数；  
 若继续执行当前任务可能导致后续清除无法完成，则立即转入剩余扫描和候选点遍历。

### `sector\_cover`

来源: `Q3\_submission/src/discrete\_model/certified\_bounds.py`, 第61—79行。

```

'''python
def sector_cover(belief):
 """At most 380 snapped points cover every possible live source.

```

A positive bearing implies local x in [0,1500], |y|<27. The 20 m rotated

```

lattice has covering radius sqrt(200); 5 m snapping adds sqrt(12.5), so
total <17.68<20. A needed point has norm <=1800+17.68<1820.
"""
near=next((p for p,kind,_ in belief.observations if kind=='near'),None)
if near is not None:return [snap(near)]
obs=next(((p,angle) for p,kind,angle in belief.observations if kind=='direction'),
None)
if obs is None:raise ValueError('Fallback clearance requires a positive
observation')
origin,angle=obs;t=math.radians(angle);e=(math.cos(t),math.sin(t));normal=(-e[1],e
[0])
points=[]
for row,y in enumerate((-40,-20,0,20,40)):
for x in (range(0,1501,20) if row%2==0 else range(1500,-1,-20)):
p=snap((origin[0]+x*e[0]+y*normal[0],origin[1]+x*e[1]+y*normal[1]))
if math.hypot(*p)<=1820 and p not in points:points.append(p)
if not points:raise ValueError('Empty sector fallback')
return points
'''

`fallback_upper`

```

来源: `Q3\_submission/src/discrete\_model/certified\_bounds.py`, 第82—95行。

```

'''python
def fallback_upper(beliefs,anchors,visited,start):
 """Fixed anchor order, then sector covers in channel order.

 Per target: incoming distance <=5120 m; full snapped serpentine <=
 7580+379*sqrt(50) m; at most 380 clears costing <=5 s each. Total <5000 s.
 Skipping outside-disk points or stopping on success only shortens the path.
 """
 k,_,nmax=counts(beliefs);unknown=sum(b.state=='unknown' for b in beliefs.values())
 pending=[p for i,p in enumerate(anchors) if i not in visited] if unknown else []
 travel=0.;p=start
 for q in pending:travel+=movement_us(p,q)/1e6+.000002;p=q
 return dict(remaining_upper_s=travel+6*unknown*len(pending)+5000*(nmax-k),
 actions_upper=unknown*len(pending)+380*(nmax-k),
 scope='executable fixed-anchor then complete-sector policy; virtual time only')
'''

`FormalController.next_job`

```

来源: `Q3\_submission/src/formal\_strategy.py`, 第27—45行。

```

'''python
def next_job(self):
 if self.reserve_locked:return self.emergency.job(self.beliefs,self.anchors,self.
 visited)
 job=super().next_job()
 unknown=sum(b.state=='unknown' for b in self.beliefs.values())
 actions=max(1,unknown) if job.kind=='scan' else 1
 operation=6*actions if job.kind=='scan' else 6 if job.kind=='measure' else 5
 step=movement_us(self.client.position,job.point)/1e6+operation
 tail=fallback_upper(self.beliefs,self.anchors,self.visited,job.point)
 self.reserve_checks+=1
 within_time=self.client.virtual_s+step+tail['remaining_upper_s']+1<=self.client.
 max_virtual_s
 within_actions=self.actions+actions+tail['actions_upper']<=self.config.max_actions
 if not (within_time and within_actions):

```

```

current=fallback_upper(self.beliefs,self.anchors,self.visited,self.client.position
)
if self.client.virtual_s+current['remaining_upper_s']>self.client.max_virtual_s:
raise ValueError('Virtual fallback reserve invariant violated')
self.reserve_locked=True
self.client.record('formal_reserve_commit',remaining_upper_s=current['
remaining_upper_s'],reason='virtual_or_action_reserve')
return self.emergency.job(self.beliefs,self.anchors,self.visited)
return job
```

```

Listing 2: Q4 算法的 Markdown 源码

1. 21站构造与全朝向覆盖证书

以有效接收范围内扫描站的凸包证明位置单元对所有朝向可检测，不以有限朝向采样代替连续覆盖证明。

`compact\_points`

来源: `Q4\_submission/src/q4/compact\_policy.py`, 第17—20行。

```

```python
def compact_points(inner=995.,outer_apothem=1801.):
outer=outer_apothem/math.cos(math.pi/12)
return [(0.,0.)]+[(round(r*math.cos(2*k*math.pi/n),6),round(r*math.sin(2*k*math.pi
/n),6))
for n,r in ((8,inner),(12,outer)) for k in range(n)]
```

```

`CompactMap.\_\_init\_\_`

来源: `Q4\_submission/src/q4/compact\_policy.py`, 第26—55行。

```

```python
def __init__(self,points=None,bins=72):
self.anchors=compact_points() if points is None else list(map(tuple,points))
cert=certify_layout(self.anchors,minimum_cell=.5,maximum_cells=40000)
if not cert['certified']:raise ValueError('Compact layout has unresolved
continuous coverage cells')
groups={}
for tri,witness in zip(cert['vertices'],cert['witnesses']):
groups.setdefault(tuple(witness),[]).extend(tri)
cells=[];owners=[];self.parents=[];max_range=0.
Merge equal-witness leaves into their convex hull. Every new hull
vertex was already certified. Both range disks and the witness hull
are convex, so the larger merged region remains certified as well.
for witness,vertices in sorted(groups.items()):
unique=np.unique(np.asarray(vertices),axis=0);hull=ConvexHull(unique)
polygon=unique[hull.vertices];scanners=np.asarray([self.anchors[i] for i in
witness])
distance=float(np.max(np.linalg.norm(polygon[:,None]-scanners[None],axis=2)))
eq=ConvexHull(scanners).equations
if distance>=999.99 or np.any(polygon@eq[:,2].T+eq[:,2]>=-1e-6):
raise ValueError('Merged witness region failed independent range/hull check')
max_range=max(max_range,distance);owner=len(self.parents);self.parents.append(
witness)
for j in range(1,len(polygon)-1):
cells.append((polygon[0],polygon[j],polygon[j+1]));owners.append(owner)
self.vertices=np.asarray(cells);self.owners=np.asarray(owners);self.bins=bins
self.full=(1<<(len(cells)*bins))-1;self.negative_mask=0;self.scanned=[];self.

```

```

 visited=set();self.cache={}
self.parent_masks=[self.bits(np.repeat((self.owners==i)[:,None],bins,axis=1)) for
 i in range(len(self.parents))]
self.certificate=dict(method='adaptive continuous triangle coverage +
 independently checked merged in-range convex hull witnesses',
source_domain_radius_m=1800,minimum_receive_radius_m=1000,anchor_count=len(self.
 anchors),
outer_apothem_m=float(min(-ConvexHull(np.asarray(self.anchors)).equations[:,2])),
maximum_witness_range_m=max_range,unmerged_certificate_cells=len(cert['vertices'])
 ,
merged_cells=len(cells),witness_groups=len(self.parents),continuous_all_headings=
 True,
global_minimum_site_count_proved=False)
``,``,`

```

### `certify\_layout`

来源: `Q4\_submission/src/q4/layout.py`, 第26—79行。

```

`python
def certify_layout(points, radius=1800., receive=1000., sides=96,
minimum_cell=.1, maximum_cells=500000):
 """Sufficient continuous certificate; failure means unverified, not infeasible.

 All range checks reserve 1 cm. Convex-hull halfspaces reserve 1 micrometre.
 Certificate leaves store the subset of scanner IDs proving that leaf.
 Returned coordinates are the actual rounded API coordinates.
 """
 points=np.asarray(points, dtype=float)
 a=np.arange(sides)*2*math.pi/sides
 boundary=np.stack((np.cos(a),np.sin(a)),axis=1)*(radius+.001)/math.cos(math.pi/
 sides)
 todo=np.array([[np.zeros(2),boundary[k],boundary[(k+1)%sides]] for k in range(
 sides)])
 leaves=[]; witnesses=[]; unresolved=[]; rounds=0; hulls={}
 while len(todo):
 rounds+=1
 safe=np.all(np.linalg.norm(todo[:,None,:,:]-points[None,:,None,:],axis=3)<receive
 -.01,axis=2)
 groups={}
 for i,row in enumerate(safe):groups.setdefault(tuple(np.flatnonzero(row)),[]).
 append(i)
 passed=np.zeros(len(todo),dtype=bool)
 for selected,ids in groups.items():
 if len(selected)<3:continue
 if selected not in hulls:
 p=points[list(selected)]
 if np.linalg.matrix_rank(p-p[0])<2:hulls[selected]=None
 else:
 hull=ConvexHull(p)
 hulls[selected]=(hull.equations,tuple(int(selected[i]) for i in hull.vertices))
 entry=hulls[selected]
 if entry is None:continue
 eq, witness=entry
 values=todo[ids]@eq[:,2].T+eq[:,2]
 ok=np.all(values < -1e-6,axis=(1,2))
 for i in np.asarray(ids)[ok]:
 passed[i]=True;leaves.append(todo[i]);witnesses.append(witness)
 failed=todo[~passed]
 if not len(failed):break

```

```

edge2=np.sum((failed-np.roll(failed,-1,axis=1))**2,axis=2)
longest=np.argmax(edge2,axis=1)
small=np.max(edge2,axis=1)<=minimum_cell**2
unresolved.extend(failed[small])
failed=failed[~small];longest=longest[~small]
if len(leaves)+len(failed)*2>maximum_cells:
unresolved.extend(failed);break
children=[]
for tri,k in zip(failed,longest):
u,v,w=tri[k],tri[(k+1)%3],tri[(k+2)%3];mid=(u+v)/2
children.extend(((u,mid,w),(mid,v,w)))
todo=np.asarray(children)
return dict(certified=not unresolved,points=points.tolist(),
vertices=np.asarray(leaves).tolist(),witnesses=[list(x) for x in witnesses],
unresolved=np.asarray(unresolved).tolist(),rounds=rounds,
source_domain_radius_m=radius,minimum_receive_radius_m=receive,
range_margin_m=.01,hull_margin_m=1e-6,outer_polygon_sides=sides,
anchor_count=len(points),method='adaptive triangle / in-range convex hull')
```

```

2. 方向、距离与无信号约束

第四问的无信号可能来自距离或方向遮蔽；候选位置对应可能朝向及接收半径，不能直接套用第三问的无信号圆盘排除。

`components`

来源：`Q4\_submission/src/q4/model.py`，第22—49行。

```

```python
def components(b,g):
 """Exact piecewise range/heading volume at a fixed hypothetical position.

 Uniform range/heading and equal prior type weights are DESIGN assumptions.
 Zero-volume boundaries are irrelevant to scoring; never used to certify.
 Returns (heading_lo,heading_hi,range_lo,range_hi,omni,relative_mass).
 """
 positive=[p for p,k,_ in b.observations if k in ('direction','near')]
 if any(distance(g,p)>5+1e-7 for p,k,_ in b.observations if k=='near'):
 return []
 if any(distance(g,p)<=5 for p,k,_ in b.observations if k=='direction'):
 return []
 if any(distance(g,p)<=20 for p in b.failed_clears):return []
 low=max([1000.]+[distance(g,p) for p in positive])
 if low>=1500:return []
 negative=list(dict.fromkeys(b.negatives))
 angles=[math.atan2(p[1]-g[1],p[0]-g[0]) for p in positive+negative]
 cuts=sorted([0.,TAU,*[(a+side*math.pi/2)%TAU for a in angles for side in (-1,1)]])
 out=[]
 high=min([1500.]+[distance(g,p) for p in negative])
 if high>low:out.append((0.,TAU,low,high,True,.5*(high-low)/500))
 for a,z in zip(cuts,cuts[1:]):
 m=(a+z)/2;u=(math.cos(m),math.sin(m))
 if any((p[0]-g[0])*u[0]+(p[1]-g[1])*u[1]<0 for p in positive):continue
 high=min([1500.]+[distance(g,p) for p in negative
 if (p[0]-g[0])*u[0]+(p[1]-g[1])*u[1]>=0])
 if high>low:out.append((a,z,low,high,False,.5*(z-a)/TAU*(high-low)/500))
 return out
```

```

```
### `DirectionMap.mask`
```

来源: `Q4\_submission/src/q4/model.py`, 第94—108行。

```
```python
def mask(self,p):
 p=tuple(p)
 if p in self.cache:return self.cache[p]
 v=np.asarray(p)-self.vertices;dist=np.linalg.norm(v,axis=2)
 angles=np.arctan2(v[:, :, 1],v[:, :, 0])[:, :, None]
 center=(np.arange(self.bins)+.5)*TAU/self.bins
 delta=np.abs((angles-center+math.pi)%TAU-math.pi)
 # Minimum dot over every angle in a bin and every triangle vertex.
 # Linear dependence on source position extends this to the entire cell.
 worst=np.minimum(math.pi,delta+math.pi/self.bins)
 dot=dist[:, :, None]*np.cos(worst)
 ok=np.all(dist<999.999,axis=1)[:, None]&np.all(dot>1e-6,axis=1)
 value=self.bits(ok)
 if len(self.cache)>256:self.cache.clear()
 self.cache[p]=value;return value
```
```

3. 测量 / 清除模式的条件续接成本

`visibility`同时考虑距离、类型和朝向;按`direction`、`near`和`no_signal`分支累计条件成本。这是数值近似,不是完整精确后验。

```
### `target_mode`
```

来源: `Q4\_submission/src/q4/coupled\_policy.py`, 第22—45行。

```
```python
def target_mode(group, job, cloud):
 weights=cloud.weights/float(cloud.weights.sum())
 if job.kind=='clear':
 cost,exits,prob=clear_tail(cloud.points,weights,job.point,forced=job.point)
 return Mode(group,job,cost,exits,prob)
 p=job.point;d=np.linalg.norm(cloud.points-p,axis=1)
 visible=cloud.visibility(p) # range AND type/heading, conditioned on history
 angle=np.degrees(np.arctan2(cloud.points[:,1]-p[1],cloud.points[:,0]-p[0]))
 mean,sigma=cloud.prediction(p);branches={}
 for offset,w in ((-math.sqrt(3),1/6),(0,2/3),(math.sqrt(3),1/6)):
 observed=np.round((angle+np.clip(mean+offset*sigma,-1,1))*100)/100%360
 bins=np.floor(observed/.5).astype(int);bins[d<=5]==-2
 for bucket in np.unique(bins):
 branches.setdefault(int(bucket),np.zeros(len(d)))[:, :] += weights*visible*w*(bins==
 bucket)
 branches[-1]=weights*(1-visible)
 cost=0.;exits=[];prob=[]
 for bucket,w in branches.items():
 selected=w>1e-12;mass=float(w[selected].sum())
 if mass<=1e-12:continue
 c,e,q=clear_tail(cloud.points[selected],w[selected],p,forced=p if bucket==-2 else
 None)
 cost+=mass*c;exits.extend(e);prob.extend(mass*q)
 prob=np.asarray(prob)
 if abs(float(prob.sum())-1)>1e-8:raise ValueError('Q4 observation partition lost
 mass')
 return Mode(group,job,cost,np.asarray(exits),prob)
```
```

4. 带覆盖与连通约束的开放路径MILP

这段处理扫描站选择、必须访问目标、覆盖证书和连通割，并保留已有可行路线。函数默认剩余资源不是部署值：实际控制器传入0.5秒。

`solve\_path`

来源：`Q4\_submission/src/q4/cover\_route.py`，第55—158行。

```
```python
def solve_path(sites,start,service,required,coverage,certificates,baseline,budget
 =.75,relative_gap=.005,primal_improvement=False):
 """Optional-vertex open path. coverage rows list direct site witnesses and
 certificate indices. A certificate can be activated only if every listed
 site is visited. baseline is independently known feasible by the caller.

 degree(i)=2*y_i-e_i; sum(e_i)=1; degree(start)=1. Subtour separation uses
 sum(x over cut(C)) >= y_k, k in C, for components not containing start.
 """
 if not math.isfinite(budget) or budget<0:raise ValueError('Invalid planning budget
 ')
 if not math.isfinite(relative_gap) or not 0<=relative_gap<1:raise ValueError('
 Invalid relative gap')
 started=time.monotonic();n=len(sites)
 if not n:return [],dict(upper_bound_s=0.,lower_bound_s=0.,gap=0.,accepted=False,
 rounds=0)
 edges=[(i,j) for i in range(n+1) for j in range(i+1,n+1)]
 ne=len(edges);yo=ne;eo=yo+n;zo=eo+n;size=zo+len(certificates)
 positions=[s.point for s in sites]+[start]
 c=np.zeros(size);c[:ne]=[math.dist(positions[i],positions[j])/5 for i,j in edges];
 c[yo:eo]=service
 rows=[];lower=[];upper=[]
 def add(terms,lo,hi):rows.append(terms);lower.append(lo);upper.append(hi)
 incident=[[] for _ in positions]
 for k,(i,j) in enumerate(edges):incident[i].append(k);incident[j].append(k)
 for i in range(n):
 add({**{k:1. for k in incident[i]},yo+i:-2.,eo+i:1.},0,0)
 add({eo+i:1.,yo+i:-1.},-np.inf,0)
 add({k:1. for k in incident[n]},1,1)
 add({eo+i:1. for i in range(n)},1,1)
 for i in required:add({yo+i:1.},1,1)
 for k,ids in enumerate(certificates):
 for i in ids:add({zo+k:1.,yo+i:-1.},-np.inf,0)
 for direct,witness in coverage:
 add({**{yo+i:1. for i in direct},**{zo+k:1. for k in witness}},1,np.inf)
 best=list(baseline);upper_bound=path_cost(sites,best,start,service);dual=0.;rounds
 =0;cuts=0;status=None
 repairs=0;primal_savings=0.
 if primal_improvement:
 candidate=improve_primal(sites,start,service,required,coverage,certificates,best)
 value=path_cost(sites,candidate,start,service)
 if value<upper_bound-1e-7:
 primal_savings+=upper_bound-value;best=candidate;upper_bound=value
 # A cutoff is safe: the supplied incumbent itself satisfies this model.
 add({i:float(v) for i,v in enumerate(c) if v},-np.inf,upper_bound+1e-6)
 while time.monotonic()-started<budget:
 rr=[];cc=[];vv=[]
 for r,terms in enumerate(rows):
 for col,v in terms.items():rr.append(r);cc.append(col);vv.append(v)
```

```

matrix=coo_matrix((vv,(rr,cc)),shape=(len(rows),size)).tocsc()
remaining=budget-(time.monotonic()-started)
if remaining<=0:break
result=milp(c,integrality=np.ones(size),bounds=Bounds(np.zeros(size),np.ones(size)
),
constraints=LinearConstraint(matrix,np.array(lower),np.array(upper)),
options=dict(time_limit=remaining,mip_rel_gap=relative_gap))
rounds+=1;status=int(result.status)
bound=getattr(result,'mip_dual_bound',None)
if bound is not None and math.isfinite(bound):dual=max(dual,float(bound))
if result.x is None:break
x=result.x
Independently reject tolerance/integrality/linear infeasibility.
rounded=np rint(x)
if np.max(np.abs(x-rounded))>1e-5:break
values=matrix@rounded
if np.any(values<np.array(lower)-1e-5) or np.any(values>np.array(upper)+1e-5):
 break
chosen={i for i in range(n) if rounded[yo+i]};adj={i:[] for i in chosen|{n}}
for k,(i,j) in enumerate(edges):
 if rounded[k]:adj.setdefault(i,[]).append(j);adj.setdefault(j,[]).append(i)
todo=set(adj);components=[]
while todo:
 component=set();stack=[min(todo)]
 while stack:
 i=stack.pop()
 if i in component:continue
 component.add(i);stack.extend(j for j in adj[i] if j not in component)
 todo-=component;components.append(component)
disconnected=[g for g in components if n not in g]
if primal_improvement:
 # An integer disconnected solution still has a valid chosen set.
 # Reconnecting its visits produces an executable incumbent before
 # the next connectivity-cut solve can exhaust its time budget.
 candidate=improve_primal(sites,start,service,required,coverage,certificates,sorted
 (chosen))
 repairs+=1;value=path_cost(sites,candidate,start,service)
 if value<upper_bound-1e-7:
 primal_savings+=upper_bound-value;best=candidate;upper_bound=value
 if not disconnected:
 order=[];previous=None;current=n
 while True:
 following=[i for i in adj[current] if i!=previous]
 if not following:break
 if len(following)!=1:raise ValueError('Invalid connected open path')
 previous,current=current,following[0];order.append(current)
 if len(order)>n:raise ValueError('Unexpected cycle')
 if set(order)!=chosen:raise ValueError('Path missed selected nodes')
 value=path_cost(sites,order,start,service)
 if value<upper_bound-1e-7:best=order;upper_bound=value
 break
 for component in disconnected:
 boundary={k:1. for k,(i,j) in enumerate(edges) if (i in component)!=(j in
 component)}
 for i in component:add(**boundary,yo+i:-1.),0,np.inf);cuts+=1
 # A dual above a known feasible incumbent is a numerical/model error, not
 # a global optimality certificate. Preserve the incumbent and flag it.
 valid_bound=dual<=upper_bound+1e-5
 return best,dict(upper_bound_s=upper_bound,lower_bound_s=dual if valid_bound else
 None,

```

```
gap=max(0.,(upper_bound-dual)/max(upper_bound,1.)) if valid_bound else None,
accepted=best!=list(baseline),rounds=rounds,connectivity_cuts=cuts,status=status,
elapsed_s=time.monotonic()-started,bound_valid=valid_bound,
requested_time_limit_s=budget,requested_relative_gap=relative_gap,
primal_repairs=repairs,primal_savings_s=primal_savings,
scope='finite frozen covering-path MILP; not full online expected time')
```

```

5. 候选执行方案与访问次序联合MILP

每组选择一种模式，流守恒和次序约束排除子回路；后续频道成本用松弛构造下界，候选路线按真实频道序列重算。V3保留原当前目标局部动作。

`path\_cost`

来源：`Q4\_submission/src/q4/mode\_milp.py`，第19—24行。

```
```python
def path_cost(problem,path):
 last=-1;q=problem.receiver;cost=0.
 for i in path:
 cost+=float((problem.first if last<0 else problem.move[last])[i]+problem.service[i
 ,q-1])
 q=int(problem.receiver_out[i,q-1]);last=i
 return cost
```

```

`solve`

来源：`Q4\_submission/src/q4/mode\_milp.py`，第27—102行。

```
```python
def solve(problem,reference_first,reference_order,seconds=2.,incumbent_path=None):
 started=time.monotonic();n=problem.n;m=len(problem.modes)
 allowed_first=set(getattr(problem,'allowed_first',range(m)))
 if reference_first not in allowed_first:raise ValueError('Incumbent first action
 must remain allowed')
 original=problem.fixed_order(reference_order,reference_first)
 free=problem.fixed_order(reference_order)
 best=min(original,free,key=lambda x:x[0]) if free[1][0] in allowed_first else
 original
 if incumbent_path is not None:
 if len(incumbent_path)!=n or len({problem.modes[i].group for i in incumbent_path})
 !=n:
 raise ValueError('Retained incumbent must cover every group exactly once')
 if incumbent_path[0] not in allowed_first:raise ValueError('Retained first action
 is not allowed')
 incumbent=(path_cost(problem,incumbent_path),tuple(incumbent_path))
 best=min(best,incumbent,key=lambda x:x[0])
 if n<2 and len(allowed_first)==m:
 return problem.solve(reference_first,reference_order)
 group=[mode.group for mode in problem.modes]
 # -1 is source, -2 is sink. No source->sink edge when work remains.
 edges=[(-1,j) for j in range(m)]+[(i,-2) for i in range(m)]
 edges += [(i,j) for i in range(m) for j in range(m) if group[i]!=group[j]]
 e=len(edges);cost=np.zeros(e+n)
 rows=[];cols=[];vals=[];lower=[];upper=[]
 def row(items,lo,hi):
 r=len(lower);lower.append(lo);upper.append(hi)
 for j,v in items:rows.append(r);cols.append(j);vals.append(v)
```

```

```

incoming=[[] for _ in range(m)];outgoing=[[] for _ in range(m)]
start=[];end=[];arcs=[]
for k,(i,j) in enumerate(edges):
    if i<0:
        start.append(k);incoming[j].append(k)
        cost[k]=problem.first[j]+problem.service[j,problem.receiver-1]
    elif j<0:
        outgoing[i].append(k);end.append(k)
    else:
        outgoing[i].append(k);incoming[j].append(k);arcs.append((k,i,j))
        cost[k]=problem.move[i,j]+problem.service[j].min()
        row([(k,1.) for k in start],1.,1.);row([(k,1.) for k in end],1.,1.)
        for g in range(n):row([(k,1.) for i in problem.indices[g] for k in incoming[i]
            ],1.,1.)
        for i in range(m):row([(k,1.) for k in incoming[i]]+[(k,-1.) for k in outgoing[i]
            ],0.,0.)
        for k,i,j in arcs:row([(k,float(n)),(e+group[i],1.), (e+group[j],-1.)],-np.inf,
            float(n-1))
matrix=coo_matrix((vals,(rows,cols)),shape=(len(lower),e+n)).tocsc()
variable_upper=np.r_[np.ones(e),np.full(n,n-1)]
for k,(i,j) in enumerate(edges):
    if i==1 and j not in allowed_first:variable_upper[k]=0.
    result=milp(cost,integrality=np.r_[np.ones(e),np.zeros(n)],
        bounds=Bounds(np.zeros(e+n),variable_upper),
        constraints=LinearConstraint(matrix,np.asarray(lower),np.asarray(upper)),
        options={'time_limit':max(.001,float(seconds)),'mip_rel_gap':.0001})
    candidate_valid=False;relaxed_primal=None
    if result.x is not None:
        selected=[edges[k] for k in range(e) if result.x[k]>.5]
        successor=dict(selected);path=[];current=-1
        for _ in range(n+1):
            current=successor.get(current,-3)
            if current<0:break
            path.append(current)
            candidate_valid=(current==n-2 and len(path)==n and len({group[i] for i in path})==n
                and len(selected)==n+1 and path[0] in allowed_first)
            if candidate_valid:
                candidate=(path_cost(problem,path),tuple(path))
                if candidate[0]<best[0]:best=candidate
                relaxed_primal=float(result.fun)
                lower_bound=problem.lower((1<n)-1,-1)
                dual=getattr(result,'mip_dual_bound',None)
                bound_valid=dual is None or not math.isfinite(float(dual)) or float(dual)<=best
                    [0]+1e-6
                if dual is not None and math.isfinite(float(dual)) and bound_valid:
                    lower_bound=max(lower_bound,min(float(dual),best[0]))
                if lower_bound>best[0]+1e-6:raise ValueError('Catalogue lower bound exceeds
                    retained feasible plan')
                assert best[0]<=original[0]+1e-6
            return best[1],dict(reference_s=float(original[0]),selected_s=float(best[0]),
                lower_s=float(lower_bound),
                gap_s=float(max(0.,best[0]-lower_bound)),exact_catalog=bool(best[0]-lower_bound<=1
                    e-6),
                expanded=int(getattr(result,'mip_node_count',0) or 0),groups=n,modes=m,
                solver='joint_service_mode_open_route_MILP',solver_status=int(result.status),
                solver_message=result.message,receiver_relaxed=True,relaxed_primal_s=
                    relaxed_primal,
                candidate_valid=bool(candidate_valid),bound_valid=bool(bound_valid),elapsed_s=time
                    .monotonic()-started,
                allowed_first_actions=len(allowed_first),

```

```
numerical_tolerance_only=True,scope='finite frozen service modes; not complete
online time bound')
...
```

```
### `CoupledPathController.configure_root`
```

来源: `Q4\_submission/src/q4/coupled\_policy.py`, 第174—181行。

```
```python
def configure_root(self,problem,first):
The clearance-only mode tails approximate the multi-measurement Q4
local policy poorly. Retain that policy for the incumbent target;
use joint modes to decide whether a DIFFERENT task should precede it.
This is a smaller explicit catalogue, not a claimed robustness bound.
group=problem.modes[first].group
if problem.modes[first].job.kind in ('measure','clear'):
problem.allowed_first={i for i,m in enumerate(problem.modes) if i==first or m.
group!=group}
...
```

```
6. 25 m有限清除遍历
```

概率信息用于排列有限清除网格的尝试顺序。

```
`Q4Controller.fallback`
```

来源: `Q4\_submission/src/q4/controller.py`, 第41—58行。

```
```python
def fallback(self,b,current):
self.statistics['geometric_fallbacks']+=1
if b.fallback_cache is None:b.fallback_cache=fallback_grid(b.polygon,25)
# 25/sqrt(2)<20: a finite covering of the entire conservative polygon.
remaining=[p for p in b.fallback_cache if all(distance(p,q)>.01 for q in b.
failed_clears)]
if not remaining:raise ValueError('Q4 conservative optical cover exhausted')
try:
cloud=self.posterior.cloud(b)
points=np.asarray(remaining)
mass=(np.linalg.norm(points[:,None,:]-cloud.points[None,:,:],axis=2)<=20)@cloud.
weights
cost=np.linalg.norm(points-current,axis=1)/5+3
if mass.max()>0:
# Change only the ORDER of the finite conservative covering.
# A depleted/misspecified cloud can never delete a clear site.
point=remaining[int(np.argmax(mass/cost))]
return Job('clear',point,b.channel,False)
except PosteriorUnavailable:pass
return Job('clear',min(remaining,key=lambda p:distance(current,p)),b.channel,False
)
...
```